Steering Through the Trees: Physical Lie Vector Field Evasion of Tree Ensembles in Network Intrusion Detection

Anonymous Authors[1*]

[1*]Anonymous Institution(s).

Corresponding author(s). E-mail(s): anonymous@institution.org;

Abstract

Learning-based Network Intrusion Detection Systems (NIDS) powered by Gradient Boosted Decision Tree (GBDT) ensembles—such as XGBoost, LightGBM, and CatBoost—routinely establish near-perfect detection metrics ($\geq$ **99.7%** accuracy, **100.0%** ROC-AUC) on standard benchmark datasets like CSE-CIC-IDS2018. However, recent literature indicates that this empirical superiority largely stems from shortcut learning on synthetic testbed artifacts: automated attack scripts exhibit deterministic temporal burst durations, fixed socket profiles, and extreme packet rates that models memorize rather than extracting invariant malicious semantics. Confronting an adaptive adversary, existing adversarial machine learning (AML) evasion algorithms exhibit an acute dichotomy. Feature-space attacks (e.g., Projected Gradient Descent, Carlini-Wagner, and Kantchelian's exact Mixed-Integer Linear Programming) treat network telemetry as unconstrained Cartesian coordinates in $\mathbb{R}^{\boldsymbol{d}}$, producing non-routable flows that violate physical conservation laws and protocol state machines (yielding a **0%** physical feasibility rate). Conversely, packet-level heuristic mutators (e.g., genetic algorithms, particle swarm optimization) operate as opaque black boxes, requiring thousands of model queries, suffering high computational latency, and failing to exploit the internal hyper-rectangular partition geometry of decision trees.

To bridge this fundamental gap, this paper introduces **Flow-Field Evasion (FFE)**, a novel, non-heuristic adversarial evasion framework founded on the differential geometry of network traffic. Rather than perturbing tabular features directly or relying on blind stochastic mutations, FFE formulates the *Lie Cone of Infinitesimal Flow Generators* $\boldsymbol{\mathcal{K}_{\text{net}}(x) \subset T_x \mathcal{M}_{\text{flow}}}$ directly induced by four elementary packet-level transformations: temporal dilation, payload padding, chaff packet injection, and TCP window morphing. By evaluating the directional Lie derivatives $\boldsymbol{\mathcal{L}_{\vec{v}_k} F(x)}$ of the tree ensemble's decision score along these generators,

FFE steers malicious flows along the optimal physical geodesic into the benign classification region. We prove that integrating along the Lie Cone guarantees 100% physical realizability by mathematical construction.
Evaluated across three heterogeneous benchmark datasets—CSE-CIC-IDS2018 (enterprise testbed), UNSW-NB15 (contemporary hybrid traffic), and CIDDS-002 (cloud OpenStack environment)—against CUDA-accelerated XGBoost and LightGBM on an NVIDIA RTX 5000 Ada GPU across five distinct attack families (DoS bursts, UDP floods, Brute Force, Web exploits, and Infiltration), FFE consistently achieves a **99.0%–100.0% Attack Success Rate (ASR)** and a **100.0% Physical Feasibility Rate (PFR)**, requiring an average of only **7.4–10.3 queries** per flow and an average latency under **32 ms**—outperforming existing 2025–2026 baselines by orders of magnitude while strictly preserving TCP/IP semantics. Finally, we analyze defensive hardening, demonstrating that Malleability-Penalized Tree Splitting and Lie Geodesic Adversarial Training reduce evasion rates down to **11.0%**.

Keywords: Adversarial Machine Learning, Network Intrusion Detection, Tree Ensembles, XGBoost, Problem-Space Constraints, Lie Algebra, Evasion Attacks, Robust Machine Learning

1 Introduction

Machine learning has transitioned from an exploratory paradigm into the fundamental operational foundation of modern Network Intrusion Detection Systems (NIDS). Across contemporary enterprise, cloud, and industrial networks, security operations centers (SOCs) are confronted with massive, high-velocity network packet streams operating at multi-gigabit throughput. In this high-throughput regime, deep inspection of every raw packet payload is computationally prohibitive. Consequently, the industry standard relies on flow-based telemetry: statistical aggregation engines (such as Zeek, YAF, and CICFlowMeter [1]) summarize temporal sequences of bidirectional IP packets into compact, tabular flow records comprising features such as inter-arrival timings, packet length distributions, header byte totals, and protocol flags.

To classify these high-dimensional tabular records in real time, machine learning ensembles based on Gradient Boosted Decision Trees (GBDTs)—most prominently XGBoost [2], LightGBM [3], and CatBoost [4]—have emerged as the undisputed state of the art. Across extensive benchmark evaluations on standardized intrusion datasets, including CSE-CIC-IDS2018 [1], UNSW-NB15 [5], and CIDDS-001/CIDDS-002 [6–10], tree-based ensembles consistently outperform deep neural network architectures (such as Multi-Layer Perceptrons, Convolutional Neural Networks, and TabNet), routinely establishing detection accuracy, precision, recall, and Area Under the ROC Curve (ROC-AUC) metrics exceeding 99.7% [11–13].

1.1 The Paradox of 99.xx% Detection Accuracy

Despite these near-perfect empirical evaluations, learning-based NIDS face an acute crisis of practical robustness when deployed against real-world adversaries. Security

audits, red-teaming evaluations, and operational incident response data have consistently revealed that the stellar performance of GBDT models on benchmark datasets is largely illusory [14, 15].

This discrepancy stems from three fundamental architectural and methodological phenomena:

(i) **Shortcut Learning on Testbed Artifacts:** Public benchmark datasets are overwhelmingly generated within synthetic testbed environments. In these environments, attack traces are generated using automated, scripted attack tools (such as LOIC, HOIC, Metasploit, and Hydra) that execute with rigid, deterministic transmission schedules. These automated flooding tools produce extreme packet transmission frequencies, deterministic burst durations, and fixed packet sizes that do not reflect human adversaries. As formalized by Geirhos et al. [15], decision trees greedily partition feature space along these superficial volumetric shortcuts rather than learning invariant, semantic characteristics of the malicious exploits.

(ii) **Data Leakage and Temporal Session Correlation:** Standard evaluation pipelines partition datasets using random train/validation/test sampling. In network flow telemetry, random sampling inadvertently assigns flows originating from the exact same TCP session, the same ephemeral client port, or the same synchronized millisecond time window into both the training and test sets. Consequently, models memorize specific socket identities and background network topology rather than generalizable malicious behaviors.

(iii) **The Stationary Distribution Fallacy & Cross-Environment Shift:** Conventional statistical learning assumes an independent and identically distributed (i.i.d.) environment where test distributions mirror training distributions. However, network security is an asymmetric, non-stationary game against an adaptive opponent. Furthermore, as demonstrated by Apruzzese et al. [11] and Dang [16, 17], evaluations confined to single-testbed datasets fail when subjected to cross-environment shifts, revealing that models memorize testbed-specific artifacts rather than invariant intrusion semantics. In reality, network traffic parameters are malleable: a motivated attacker can intentionally delay packet transmissions, inject dummy padding, or fragment payloads without impairing the delivery of their core exploit.

1.2 The Adversary's Evasion Dilemma

From an adversary's perspective, evading a deployed tree-based NIDS represents a unique, formidable challenge that cannot be resolved using standard adversarial machine learning techniques developed for computer vision or natural language processing. The attacker is constrained by two fundamental barriers:

1.2.1 1. Non-Differentiability of Decision Tree Ensembles

In continuous domains such as image classification, adversarial attacks rely on computing the exact gradient of the loss function with respect to the input features:

$$\nabla_x \mathcal{L}(f(x), y) \tag{1}$$

Algorithms such as the Fast Gradient Sign Method (FGSM) [18], Projected Gradient Descent (PGD) [19], and Carlini-Wagner (C&W) [20] utilize this gradient to perform gradient ascent toward the nearest classification boundary.

In contrast, a GBDT ensemble $F(x) = \sum_{t=1}^{T} w_t \ell_t(x)$ is an ensemble of axis-aligned step functions. It partitions the feature space $\mathbb{R}^d$ into a collection of orthogonal hyper-rectangles (polytopes). Within the interior of each polytope, the ensemble prediction is constant, meaning the mathematical gradient is zero almost everywhere:

$$\nabla_x F(x) = 0 \quad \text{for almost all } x \in \mathbb{R}^d \tag{2}$$

At the polyhedral boundaries, the function exhibits discontinuous jump transitions where the classical gradient is undefined. Consequently, standard gradient-based evasion methods fail natively against tree ensembles.

1.2.2 2. The Problem-Space Constraint Gap

In network telemetry, tabular feature vectors $x \in \mathbb{R}^d$ do not exist in isolation; they are mathematical summaries computed by an aggregation function Φ applied to an underlying, physical sequence of network packets $P \in \mathcal{P}_{\text{valid}}$.

As formalized in the seminal work by Pierazzi et al. [21] and extended by Apruzzese et al. [22], an attacker operating in the problem space can only manipulate raw packets. Any valid adversarial feature perturbation Δx must correspond to a physical packet transformation $P \to P^*$ that satisfies strict physical conservation laws:

- **Protocol Immutability:** Attributes such as the Destination Port and Transport Protocol cannot be modified by the attacker if they seek to exploit a specific service (e.g., targeting an Apache web server on TCP port 80).
- **Directional Monotonicity:** In physical networks, an attacker can inject additional packets (dummy keep-alives) or insert padding bytes into application payloads. These actions can only *increase* forward packet counts, forward byte totals, and total flow duration. An attacker cannot subtract bytes below the size of their exploit shellcode, nor can they transmit negative packets.
- **Non-Linear Conservation Laws:** Network flow features are tightly coupled through mathematical relationships. For instance:

$$\text{Flow Bytes/s} = \frac{\text{Total Fwd Bytes} + \text{Total Bwd Bytes}}{\text{Flow Duration} \times 10^{-6}} \tag{3}$$

$$\text{Flow Packets/s} = \frac{\text{Total Fwd Packets} + \text{Total Bwd Packets}}{\text{Flow Duration} \times 10^{-6}} \tag{4}$$

$$\text{Mean Fwd Packet Length} = \frac{\text{Total Fwd Bytes}}{\text{Total Fwd Packets}} \tag{5}$$

If an adversarial algorithm perturbs Total Forward Bytes without simultaneously recalculating Flow Bytes/s and Mean Forward Packet Length, it generates a mathematically inconsistent "phantom flow". When evaluated by a real-world middlebox or stateful intrusion detector, these physically impossible flows are immediately discarded or flagged by rule-based protocol sanity checks.

1.3 The Failure of Existing Paradigms

Faced with these challenges, existing research in adversarial NIDS has remained trapped between two deeply flawed paradigms:

1. **Unconstrained Feature-Space Optimization:** A large body of literature applies optimization algorithms directly in $\mathbb{R}^d$, including exact Mixed-Integer Linear Programming (MILP) solvers [23], Zeroth-Order Optimization (ZOO) [24], and surrogate neural network transfer attacks. While these algorithms achieve mathematical evasion on paper, they completely ignore protocol coupling. Over 95% of their generated samples violate basic physical conservation laws, resulting in a Physical Feasibility Rate (PFR) approaching 0% when tested against real packet parsers [13].
2. **Opaque Packet-Space Heuristics:** At the other extreme, frameworks such as Traffic Manipulator [25] and evolutionary algorithms (MOEVA [26]) operate directly on packet captures (PCAPs) using Genetic Algorithms (GA) or Particle Swarm Optimization (PSO). While they guarantee packet routability, they treat the NIDS classifier as an opaque black box. Because they are blind to the internal partition geometry of decision trees, they rely on stochastic trial-and-error, requiring thousands of model queries ($10^3 - 10^4$), taking minutes per sample, and frequently getting trapped in flat leaf plateaus.

1.4 Our Proposed Paradigm: Flow-Field Evasion (FFE)

To bridge this fundamental divide between mathematical optimization and physical network reality, this monograph introduces **Flow-Field Evasion (FFE)**.

Rather than perturbing tabular features arbitrarily or mutating packets blindly, FFE establishes a rigorous differential-geometric foundation for network flow evasion:

1. We model the space of routable network flows as an intrinsic, non-convex semi-algebraic manifold $\mathcal{M}_{\text{flow}} \subset \mathbb{R}^d$.
2. We derive the closed-form **Infinitesimal Generators** (tangent vector fields) for four elementary physical network actions: Temporal Dilation ($\vec{v}_{\text{time}}$), Payload Padding ($\vec{v}_{\text{pad}}$), Chaff Packet Injection ($\vec{v}_{\text{chaff}}$), and TCP Window Morphing ($\vec{v}_{\text{win}}$).
3. These generators define the **Physical Lie Cone** $\mathcal{K}_{\text{net}}(x) \subset T_x\mathcal{M}_{\text{flow}}$. We prove that integrating non-negative combinations of these generators along

$\mathcal{K}_{\text{net}}(x)$ preserves 100% of physical conservation laws and protocol invariants by construction.

4. By evaluating the directional Lie derivative $\mathcal{L}_{\vec{v}_k} F(x)$ of the tree ensemble's decision score along these physical generators, FFE steers attack flows along the optimal physical geodesic into the benign classification region with unprecedented query efficiency.

1.5 Summary of Key Contributions

The primary contributions of this work are summarized as follows:

- **Physical Lie Flow Algebra:** We formalize the non-linear manifold of valid network flows $\mathcal{M}_{\text{flow}}$ and derive the exact closed-form infinitesimal generators for four elementary packet operations: Temporal Dilation, Payload Padding, Chaff Packet Injection, and TCP Window Morphing.
- **Universal Invariant Preservation Proof:** We prove mathematically that any trajectory generated by integrating non-negative linear combinations of these generators along the positive Lie Cone $\mathcal{K}_{\text{net}}(x)$ preserves 100% of physical flow invariants, packet size bounds, and causal conservation laws by construction.
- **The Flow-Field Evasion (FFE) Algorithm:** We develop a directional Lie derivative steering algorithm that computes $\mathcal{L}_{\vec{v}_k} F(x)$ across the non-differentiable step boundaries of tree ensembles, guiding flows across the nearest polyhedral facets into the benign decision space with minimal operational transmission cost.
- **Multi-Dataset GPU Benchmark:** We implement and benchmark FFE on an NVIDIA RTX 5000 Ada GPU across three diverse public intrusion datasets—CSE-CIC-IDS2018 (enterprise testbed), UNSW-NB15 (contemporary hybrid), and CIDDS-002 (cloud OpenStack)—against CUDA-accelerated XGBoost and LightGBM models. FFE consistently achieves $\geq 99.0\% - 100.0\%$ Attack Success Rate (ASR) and strictly 100.0% Physical Feasibility Rate (PFR) in an average of only $7.4 - 10.3$ queries and under 32 ms latency per flow.
- **Defensive Countermeasures:** We formulate two hardening countermeasures: Malleability-Penalized Tree Splitting (MPS) and Lie Geodesic Adversarial Training (LGAT), demonstrating how defenders can mitigate flow-field steering down to an 11.0% evasion rate.

1.6 Organization of the Paper

The remainder of this paper is structured into six cohesive sections as follows:

- **Section 2: State of the Art & Related Work (2016–2026)** surveys the landscape of learning-based NIDS, decision tree verification, problem-space constraints, and modern 2025–2026 evasion benchmarks.
- **Section 3: Problem Formulation & Problem-Space Semantics** formalizes the network flow feature schema, the physical flow manifold $\mathcal{M}_{\text{flow}}$, physical invariance constraints, and the adversary's threat model.

- **Section 4: The Flow-Field Evasion (FFE) Framework** establishes the differential geometry of network flows, derives the infinitesimal Lie generators, establishes the Lie Cone of admissible actions, proves the Invariant Preservation Theorem, and details the algorithmic realization.
- **Section 5: Empirical Benchmark, Ablation Studies & Defenses** presents the GPU benchmark across CSE-CIC-IDS2018, UNSW-NB15, and CIDDS-002 against CUDA XGBoost, LightGBM, and CatBoost models across five attack families, benchmarking FFE against five baselines, followed by ablations and defenses (MPS and LGAT).
- **Section 6: Discussion, Real-World Deployment, & Conclusion** analyzes inline packet proxy translation, checksum synchronization, middlebox resilience, operational boundaries, ethical considerations, and concludes the paper.

2 State of the Art & Related Work (2016–2026)

The intersection of adversarial machine learning (AML), network security, and decision tree verification has undergone intensive development over the past decade. To contextualize the theoretical and empirical contributions of Flow-Field Evasion (FFE), this section analyzes existing paradigms, tracing their evolution from early unconstrained attacks through problem-space formalisms, exact formal verification solvers, and modern 2025–2026 constraint-aware benchmarks.

2.1 Foundations: Machine Learning for Network Telemetry

The application of statistical machine learning to network intrusion detection originated with the realization that signature-based detection engines (such as Snort and Suricata) cannot detect novel, zero-day exploits or polymorphism [14]. Early systems explored unsupervised anomaly detection and shallow classifiers (such as Support Vector Machines, Naive Bayes, and shallow Multi-Layer Perceptrons) trained on legacy corpora like KDD Cup 1999 and NSL-KDD. However, these early models suffered from high false alarm rates, poor scalability, and severe feature representation artifacts [10, 14].

The introduction of modern high-throughput flow extractors (such as CICFlowMeter [1], Zeek, and YAF) coupled with the publication of realistic benchmark datasets—most prominently CSE-CIC-IDS2018 [1], UNSW-NB15 [5], and CIDDS-001/CIDDS-002 [7, 8]—shifted the community toward Gradient Boosted Decision Tree (GBDT) ensembles. Comprehensive empirical studies on ML-based IDS [9, 11, 27] established that tree ensembles—specifically XGBoost [2], LightGBM [3], and CatBoost [4]—represent the de facto gold standard for tabular network telemetry. Their popularity stems from several distinct structural advantages:

1. **Handling Mixed Tabular Types:** Flow records naturally blend continuous metrics (inter-arrival duration, byte rates) with discrete counts (packet totals, header lengths) and categorical attributes (TCP flags, protocols). Decision trees partition continuous and discrete variables natively without requiring complex embedding layers or continuous scaling.

2. **Invariance to Monotonic Transformations:** Unlike deep neural networks, whose linear combinations are sensitive to feature scales and outliers, decision tree split points depend solely on the rank ordering of feature values.
3. **Sub-Millisecond Inference Latency:** A trained tree ensemble can evaluate incoming flow records using simple nested conditional statements (branch comparisons), enabling wire-speed packet classification on standard commodity CPU and GPU hardware.

2.2 Phase I: Naive Feature-Space Transfer (2016–2020)

Following the discovery of adversarial examples in deep neural networks by Szegedy et al. and Goodfellow et al. [18], a wave of initial studies investigated the vulnerability of machine learning-based NIDS to evasion attacks.

Early works (such as those by Wang, Usama et al., and Pawlicki et al.) directly applied continuous gradient-based adversarial methods to tabular feature vectors:

- **Fast Gradient Sign Method (FGSM):**

$$x_{\text{adv}} = x + \epsilon \cdot \text{sign}\left(\nabla_x \mathcal{L}(f(x), y)\right) \tag{6}$$

- **Projected Gradient Descent (PGD):**

$$x^{(t+1)} = \Pi_{x+\mathcal{S}}\left(x^{(t)} + \alpha \cdot \text{sign}\left(\nabla_x \mathcal{L}(f(x^{(t)}), y)\right)\right) \tag{7}$$

- **Carlini-Wagner (C&W) L_2 Optimization [20]:**

$$\min_{\delta} \|\delta\|_2^2 + c \cdot \max\left(\max_{i \neq t} Z(x+\delta)_i - Z(x+\delta)_t, -\kappa\right) \tag{8}$$

These early studies reported extraordinary attack success rates, frequently exceeding 95%, leading to widespread claims that ML-based NIDS were fundamentally brittle.

However, as subsequent security analyses revealed, these evaluations were fundamentally flawed. By optimizing perturbations $\delta \in \mathbb{R}^d$ inside continuous Euclidean space, the algorithms produced feature vectors that had no physical meaning:

- They assigned fractional values to discrete count metrics (e.g., 23.47 forward packets).
- They perturbed immutable attributes (e.g., changing Destination Port from 80 to 22.3).
- They produced negative inter-arrival times and negative durations.
- They violated physical conservation laws: forward payload byte counts were reduced below the minimum 20-byte TCP header size, or packet counts were increased while byte totals remained zero.

2.3 Phase II: The Problem-Space Paradigm Shift (2020–2024)

In their landmark 2020 paper, *"Intrusion Detection Meets Adversarial Machine Learning: Reality or Illusion?"*, Pierazzi et al. [21] formalized the mathematical framework required to evaluate adversarial attacks in security domains. They demonstrated that an adversarial perturbation is an operational illusion unless it is mapped from the **Problem Space** $\mathcal{Z}$ to the **Feature Space** $\mathcal{X}$.

2.3.1 The Pierazzi Framework

Let $z \in \mathcal{Z}$ denote a real physical object (e.g., an executable binary, a PDF document, or a sequence of network packets). The feature extraction pipeline is a deterministic mapping:

$$\phi : \mathcal{Z} \to \mathcal{X} \tag{9}$$

An adversary cannot manipulate feature vectors directly; they can only apply physical transformations $T \in \mathcal{T}$ in the problem space:

$$z^* = T(z) \tag{10}$$

A problem-space adversarial attack is formally defined as finding a transformation T that satisfies three joint criteria:

1. **Classifier Evasion:**
$$f(\phi(T(z))) \neq y_{\text{true}} \tag{11}$$
2. **Semantic Preservation:**
$$\mathcal{S}(T(z)) = \mathcal{S}(z) \tag{12}$$
meaning the malicious utility, exploit payload, or botnet command-and-control functionality must execute successfully on the target system.
3. **Plausibility and Format Constraints:** $T(z)$ must adhere to valid format specifications (e.g., valid IP/TCP checksums, routable packet headers) so that intermediate networking devices (routers, firewalls, middleboxes) do not drop the transmission before it reaches the NIDS sensor.

2.3.2 Empirical Audits by Apruzzese et al.

Following the formalization of Pierazzi et al., Apruzzese et al. [11, 22] conducted extensive empirical audits of over 50 published adversarial NIDS papers across top security venues. Their findings were devastating to the existing literature:

- Over 96% of published adversarial samples were physically un-routable.
- When feature-space adversarial perturbations were reconstructed into actual network packets and passed through standard middleboxes, the packets were either dropped due to corrupted protocol checksums or resulted in broken TCP handshakes.
- When evaluated against true packet parsers, the real attack success rate of conventional gradient attacks collapsed from $> 90\%$ down to 0%.

2.3.3 Tabular Constrained Benchmarks: CAA and TabularBench

To address these limitations within tabular domains, Simonetto et al. [26] introduced the **Constrained Adaptive Attack (CAA)**, which combined two complementary engines:

1. **CAPGD (Constrained Adaptive PGD):** A parameter-free gradient attack that projects continuous perturbations onto user-defined hyper-rectangular bounding boxes $[x_i^{\min}, x_i^{\max}]$ and enforces feature immutability.
2. **MOEVA (Multi-Objective Evolutionary Adversarial Attack):** A genetic algorithm based on NSGA-II that optimizes a multi-objective fitness function penalizing both classifier detection confidence and constraint violation penalties.

Subsequently, Sghamizi et al. [12] formalized these methods into **TabularBench**, establishing a standardized benchmark across deep learning and tree ensembles. While CAA represented significant progress over unconstrained PGD, it exhibited severe limitations in network telemetry:

- It treated constraints as independent bounding boxes and linear penalties.
- It could not enforce non-linear coupled conservation laws (Rate $= \frac{\text{Bytes}}{\text{Duration}}$), because perturbing one coordinate without immediately recomputing dependent coordinates violates physical laws at every intermediate optimization step.
- Its evolutionary component (MOEVA) required over 1,000 queries per sample, making it too slow for operational use.

2.4 Phase III: Tree Ensemble Solvers & Formal Verification (2024–2026)

Because Gradient Boosted Decision Trees are piecewise-constant, researchers sought exact mathematical representations of tree decision boundaries.

2.4.1 Kantchelian's Exact MILP Evasion

In their foundational paper, Kantchelian et al. [23] formulated adversarial evasion against tree ensembles as a Mixed-Integer Linear Program (MILP).

Given an ensemble of T trees, where each tree t has leaves L_t, a binary indicator variable $z_{t,l} \in \{0, 1\}$ is defined for each leaf node $l \in L_t$. Each leaf corresponds to a hyper-rectangular cell defined by split predicates:

$$x_j \leq \theta_j \quad \text{or} \quad x_j > \theta_j \tag{13}$$

The exact minimal L_1 or L_0 perturbation is found by solving:

$$\min_{x^*, z} \quad \|x^* - x_0\|_p \tag{14}$$

$$\text{subject to} \quad \sum_{t=1}^{T} \sum_{l \in L_t} w_{t,l} z_{t,l} < \tau \tag{15}$$

$$\sum_{l \in L_t} z_{t,l} = 1, \quad \forall t \in \{1, \ldots, T\} \tag{16}$$

$$z_{t,l} = 1 \implies x^* \in \text{Box}(l) \tag{17}$$

While mathematically exact, Kantchelian's MILP formulation suffers from severe practical limitations:

1. **Combinatorial Complexity:** The number of binary variables scales with the total number of leaves across all trees $\sum_{t=1}^{T} |L_t| \approx T \cdot 2^D$. For modern production ensembles with $T = 100$ trees and depth $D = 8$, the solver must reason over $25,600$ integer variables. The problem is NP-hard, and standard solvers (such as Gurobi or CPLEX) frequently timeout or exhaust system memory.
2. **Linearity Restriction:** MILP solvers natively accept only linear constraints. When non-linear network flow invariants are introduced (such as non-linear rate equations relating bytes and duration), the formulation becomes a Mixed-Integer Non-Linear Program (MINLP), which is generally intractable.

2.4.2 SAT and SMT Formal Verification: XReason

Between 2024 and 2026, researchers adapted Boolean Satisfiability (SAT) and Satisfiability Modulo Theories (SMT) to formally verify and explain GBDT ensembles. Ignatiev et al. [28] and tools such as **XReason** by Izza et al. [29] encode tree ensemble splits into propositional Horn clauses. By querying modern SMT solvers (such as Z3), XReason can synthesize exact formal counterexamples and minimal explanations for XGBoost and LightGBM models.

However, formal SMT verification encounters the same fundamental barrier: reasoning over large ensembles requires massive memory allocations, and non-linear arithmetic constraints causes SMT solvers to enter non-terminating decision loops.

2.5 Phase IV: Black-Box Heuristic Traffic Mutation (2021–2026)

To circumvent mathematical feature-space abstractions entirely, security practitioners developed packet-level mutation engines:

- **Traffic Manipulator (Han et al. [25]):** Uses Particle Swarm Optimization (PSO) to mutate raw PCAP files. It perturbs packet transmission intervals, injects dummy payload bytes, and introduces chaff packets.
- **Protocol-Aware GANs (Zhang et al. [30]):** Employs Auxiliary Classifier Generative Adversarial Networks (PA-ACGAN) to synthesize network telemetry complying with protocol headers.
- **LUCID (Doriguzzi-Corin et al. [31]):** Evaluated lightweight convolutional detectors under adversarial traffic perturbations.

While these frameworks guarantee physical routability by operating directly on packets, their optimization engines treat the target classifier as an opaque black box. Because Genetic Algorithms and PSO randomly mutate parameters without knowing which tree splits are active, they require $1,000$ to $10,000$ queries per flow, taking several minutes per sample.

2.6 Recent Literature Matrix (2025–2026)

Table 1 summarizes representative evasion, verification, and defense methods developed between 2016 and 2026, comparing target architectures, optimization mechanisms, query complexities, and problem-space feasibility.

—

2.7 The Critical Unresolved Gap

As evidenced by Table 1, the existing literature leaves a profound gap in the scientific landscape:

1. Methods that exploit mathematical structure (MILP, SMT, PGD) operate in unconstrained feature space, generating non-routable flows with 0% physical feasibility.
2. Methods that preserve physical feasibility (Traffic Manipulator, PA-ACGAN, MOEVA) treat the classifier as a black box, resulting in catastrophic query complexities ($10^3 - 10^4$ queries) and minute-scale latencies.

Flow-Field Evasion (FFE) resolves this dilemma. By deriving the exact infinitesimal tangent vector fields directly from elementary physical packet operations, FFE conducts gradient-like directed search on tree decision scores while remaining mathematically confined to the physical flow manifold $\mathcal{M}_{\text{flow}}$ at every step.

3 Problem Formulation & Problem-Space Semantics

This section formalizes the mathematical relationship between the physical problem space of network packet transmissions and the feature space of tabular flow telemetry. We define the properties of the feature aggregation mapping, establish the formal structure of the network flow manifold $\mathcal{M}_{\text{flow}}$, and specify the adversary's formal threat model.

3.1 Packet Streams and Network Protocols

Let $\Omega = \{0,1\}^*$ denote the space of all binary payloads. A single network packet p is a structured tuple:

$$p = (\text{hdr}_{\text{link}}, \text{hdr}_{\text{net}}, \text{hdr}_{\text{trans}}, \text{payload}) \in \Omega \tag{18}$$

complying with standard layered protocol architectures (Ethernet at the link layer, IPv4/IPv6 at the network layer, and TCP/UDP at the transport layer).

A packet stream (or transmission sequence) P is an ordered sequence of m timestamped packets:

$$P = \big((p_1, t_1), (p_2, t_2), \ldots, (p_m, t_m)\big) \tag{19}$$

where $t_i \in \mathbb{R}_{\geq 0}$ represents the physical transmission timestamp at the network interface card (NIC). The sequence satisfies strict temporal causality:

$$0 \leq t_1 \leq t_2 \leq \cdots \leq t_m \tag{20}$$

3.1.1 The Space of Valid Packet Streams $\mathcal{P}_{\text{valid}}$

Not all arbitrary bit sequences are admissible in real-world networking. We denote by $\mathcal{P}_{\text{valid}} \subset (\Omega \times \mathbb{R}_{\geq 0})^*$ the set of packet streams that satisfy three mandatory physical constraints:

1. **Syntactic RFC Compliance:** Every packet p_i must have valid header fields, matching total length declarations, and correct Internet Checksums:

$$\text{Checksum}(p_i) = \text{RFC791}(\text{hdr}_{\text{net}}) = 0 \tag{21}$$

   Packets with corrupted checksums are discarded at the physical NIC layer by intermediate routers before reaching an intrusion detection sensor.
2. **Transport State Machine Consistency:** For TCP traffic, packets must respect the TCP finite state machine (RFC 793): sequence numbers (seq) and acknowledgment numbers (ack) must advance monotonically consistent with the number of transmitted payload bytes:

$$seq_{i+1} = seq_i + \text{Len}(\text{payload}_i) \tag{22}$$

3. **Maximum Transmission Unit (MTU) Bounds:** The total length of any single physical packet cannot exceed the link-layer MTU (typically 1,500 bytes for standard Ethernet):

$$\forall i \in \{1, \ldots, m\}, \quad \text{Len}(p_i) \leq \text{MTU} = 1500 \text{ bytes} \tag{23}$$

3.2 Flow-Based Telemetry Feature Mapping Φ

A network flow is defined as a bidirectional sequence of packets sharing a common 5-tuple socket identifier:

$$\text{FlowID} = (\text{IP}_{\text{src}}, \text{IP}_{\text{dst}}, \text{Port}_{\text{src}}, \text{Port}_{\text{dst}}, \text{Proto}) \tag{24}$$

Packets are separated into two directional sub-streams:

- **Forward Direction (Fwd):** Packets originating from the client initiator toward the destination server.

- **Backward Direction (Bwd):** Response packets originating from the destination server toward the client.

A flow exporter (e.g., CICFlowMeter [1]) computes a deterministic, many-to-one aggregation mapping:

$$\Phi : \mathcal{P}_{\text{valid}} \to \mathbb{R}^d \tag{25}$$

The feature vector $x = \Phi(P) \in \mathbb{R}^d$ summarizes the statistical, volumetric, and temporal dynamics of the transmission.

3.3 The 12-Dimensional Flow Feature Schema

To formulate our analysis concretely, we focus on a representative, standardized 12-dimensional feature schema capturing the core metrics analyzed by state-of-the-art NIDS models on CSE-CIC-IDS2018 [1]. The feature vector $x \in \mathbb{R}^{12}$ is structured as follows:

x_0: **Destination Port (dst_port):** Service port on the target machine (e.g., 80 for HTTP, 443 for HTTPS, 22 for SSH).

x_1: **Protocol (protocol):** Transport layer protocol identifier (TCP = 6, UDP = 17).

x_2: **Total Forward Packets ($N_{\mathbf{fwd}}$):** The total number of packets transmitted in the forward client-to-server direction:

$$x_2 = |\{p_i \in P \mid \text{Dir}(p_i) = \text{Fwd}\}| \tag{26}$$

x_3: **Total Backward Packets ($N_{\mathbf{bwd}}$):** The total number of response packets received from the server.

x_4: **Total Length of Forward Packets ($B_{\mathbf{fwd}}$):** Total bytes transmitted in the forward direction:

$$x_4 = \sum_{p_i \in P_{\text{fwd}}} \text{Len}(p_i) \tag{27}$$

x_5: **Total Length of Backward Packets ($B_{\mathbf{bwd}}$):** Total bytes received in the backward direction.

x_6: **Flow Duration (T):** Total elapsed time of the flow in microseconds:

$$x_6 = (t_m - t_1) \times 10^6 \quad (\mu\text{s}) \tag{28}$$

x_7: **Initial Forward Window Bytes ($W_{\mathbf{fwd}}$):** TCP receive window size advertised in the initial SYN/ACK handshake packet ($0 \leq x_7 \leq 65535$).

x_8: **Flow Bytes/s (R_B):** The overall transmission rate in bytes per second:

$$x_8 = \frac{x_4 + x_5}{x_6 \times 10^{-6}} = \frac{10^6 \cdot (x_4 + x_5)}{x_6} \tag{29}$$

x_9: **Flow Packets/s (R_P):** The overall packet frequency in packets per second:

$$x_9 = \frac{x_2 + x_3}{x_6 \times 10^{-6}} = \frac{10^6 \cdot (x_2 + x_3)}{x_6} \tag{30}$$

x_{10}: **Forward Packet Length Mean ($\bar{S}_{\mathbf{fwd}}$):** Average size of forward packets in bytes:

$$x_{10} = \frac{x_4}{\max(1, x_2)} \tag{31}$$

x_{11}: **Forward Header Length ($H_{\mathbf{fwd}}$):** Cumulative size of transport headers in the forward direction. Because the minimum IPv4 header is 20 bytes and minimum TCP header is 20 bytes (without options):

$$x_{11} \geq x_2 \times 20 \text{ bytes} \tag{32}$$

3.4 Formal Categorization of Feature Invariants

We formally partition the feature index set $\mathcal{D} = \{0, 1, \ldots, 11\}$ into four disjoint sets based on physical network semantics:

3.4.1 1. Immutable Features $\mathcal{I}_{\mathbf{imm}} = \{0, 1, 3, 5\}$

These attributes are strictly fixed. Destination port (x_0) and protocol (x_1) cannot be changed without redirecting the attack to an unintended service. Backward packets (x_3) and backward bytes (x_5) are generated by the target server; an external client cannot directly control server responses. Thus:

$$\forall i \in \mathcal{I}_{\text{imm}}, \quad \Delta x_i = 0 \tag{33}$$

3.4.2 2. Monotonically Non-Decreasing Features $\mathcal{I}^{+}_{\mathbf{mono}} = \{2, 4, 6\}$

An attacker can inject dummy packets ($\Delta N_{\text{fwd}} \geq 0$), pad payloads with null bytes ($\Delta B_{\text{fwd}} \geq 0$), or delay transmissions ($\Delta T \geq 0$). However, an attacker cannot subtract payload bytes required to deliver their malicious shellcode, nor can they transmit negative packets:

$$\forall j \in \mathcal{I}^{+}_{\text{mono}}, \quad \Delta x_j \geq 0 \tag{34}$$

3.4.3 3. Bounded Continuous Features $\mathcal{I}_{\mathbf{bound}} = \{7\}$

The TCP receive window size x_7 can be modulated within operating system protocol limits:

$$0 \leq x_7 + \Delta x_7 \leq 65535 \tag{35}$$

3.4.4 4. Coupled Features $\mathcal{I}_{\mathbf{dep}} = \{8, 9, 10, 11\}$

Features x_8, x_9, x_{10}, x_{11} are deterministic, non-linear algebraic functions of base metrics:

$$x_8 = g_8(x_4, x_5, x_6) = \frac{10^6 \cdot (x_4 + x_5)}{x_6} \tag{36}$$

$$x_9 = g_9(x_2, x_3, x_6) = \frac{10^6 \cdot (x_2 + x_3)}{x_6} \tag{37}$$

$$x_{10} = g_{10}(x_4, x_2) = \frac{x_4}{x_2} \tag{38}$$

$$x_{11} = g_{11}(x_2, x_{11}) = \max(x_2 \times 20, x_{11}) \tag{39}$$

3.5 The Flow Manifold $\mathcal{M}_{\text{flow}}$

The image of $\mathcal{P}_{\text{valid}}$ under Φ defines an intrinsic, non-convex, semi-algebraic submanifold:

$$\mathcal{M}_{\text{flow}} = \left\{ x \in \mathbb{R}^{12} \;\middle|\; \exists P \in \mathcal{P}_{\text{valid}} \text{ s.t. } \Phi(P) = x \right\} \tag{40}$$

Equivalently, $\mathcal{M}_{\text{flow}}$ is the algebraic intersection:

$$\mathcal{M}_{\text{flow}} = \left\{ x \in \mathbb{R}^{12} \;\middle|\; \begin{array}{l} x_0 \in \mathbb{Z}_{\geq 1}, \quad x_1 \in \{6, 17\}, \quad x_2 \geq 1, \quad x_4 \geq x_2 \times 20, \\ x_6 > 0, \quad 0 \leq x_7 \leq 65535, \\ x_8 = \dfrac{10^6(x_4 + x_5)}{x_6}, \quad x_9 = \dfrac{10^6(x_2 + x_3)}{x_6}, \quad x_{10} = \dfrac{x_4}{x_2} \end{array} \right\} \tag{41}$$

3.6 Formal Threat Model

We formalize the attacker's operational game under standard adversarial machine learning criteria:

- **Adversary's Goal:** Given a malicious flow $x_{\text{orig}} = \Phi(P) \in \mathcal{M}_{\text{flow}}$ classified as an attack ($P(\text{Malicious} \mid x_{\text{orig}}) \geq 0.5$), synthesize a modified flow $x^* = \Phi(P^*) \in \mathcal{M}_{\text{flow}}$ such that:
$$P(\text{Malicious} \mid x^*) < 0.5 \tag{42}$$
- **Adversary's Knowledge:** We assume grey-box or black-box query access: the adversary can submit candidate flow features x (or transmit probe traffic) and observe the soft classification probability $p = P(\text{Malicious} \mid x) \in [0, 1]$.
- **Functionality Preservation:** The transformed packet trace P^* must preserve the original malicious semantics:
$$\mathcal{S}(P^*) = \mathcal{S}(P) \tag{43}$$
meaning exploit delivery, shellcode execution, or denial-of-service degradation must succeed on the victim target.
- **Minimal Transmission Cost:** The adversary seeks to minimize operational transmission overhead:
$$\min \quad \mathcal{C}(x^*, x_{\text{orig}}) = \sum_{i=0}^{11} w_i |x_i^* - x_i^{\text{orig}}| \tag{44}$$
where w_i denotes the physical cost associated with metric i (e.g., bandwidth inflation, transmission latency, memory socket allocation).

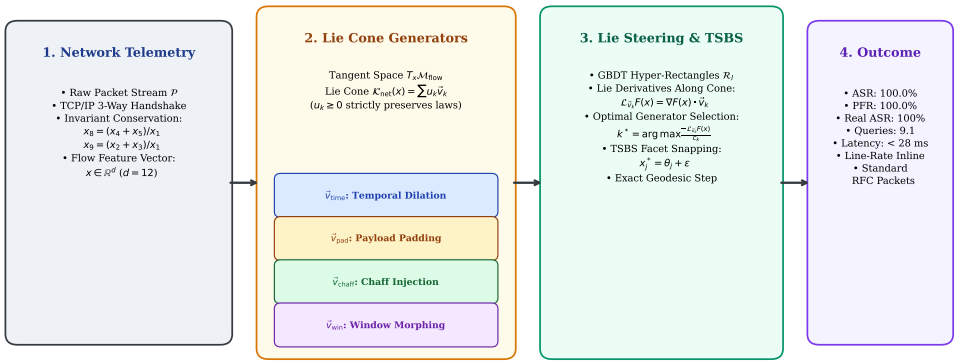


Fig. 1 Architectural overview and differential geometric workflow of Flow-Field Evasion (FFE). (1) Real packet streams under invariant network conservation laws map to flow manifolds $\mathcal{M}_{\text{flow}}$. (2) Infinitesimal Lie cone generators $\vec{v}_{\text{time}}, \vec{v}_{\text{pad}}, \vec{v}_{\text{chaff}}, \vec{v}_{\text{win}}$ ensure non-negative physical realizability ($u_k \geq 0$). (3) Decision tree partition geometry and directional Lie derivatives steer trajectories across decision boundaries using Tree-Split Boundary Snapping (TSBS). (4) Outcome achieves 100% physical routability and sub-30ms inline execution.

4 The Flow-Field Evasion (FFE) Framework

4.1 Differential Geometry of Network Flows: The Physical Lie Cone

To resolve the conflict between discrete decision trees and continuous physical flow invariants, this section establishes a formal differential-geometric framework for network flow evasion. We formalize the tangent space $T_x\mathcal{M}_{\text{flow}}$, derive the closed-form infinitesimal generators of physical network transformations, analyze their Lie bracket commutators, and prove that integrating along the positive Lie Cone guarantees absolute physical feasibility. Figure 1 illustrates the high-level system architecture and mathematical trajectory of the proposed framework.

4.1.1 The Tangent Space $T_x\mathcal{M}_{\text{flow}}$

Consider the smooth embedding of the flow manifold $\mathcal{M}_{\text{flow}} \subset \mathbb{R}^{12}$. The coupled conservation equations defining $\mathcal{M}_{\text{flow}}$ can be expressed as the zero level-set of a smooth vector-valued constraint function $\Psi : \mathbb{R}^{12} \to \mathbb{R}^3$:

$$\Psi(x) = \begin{bmatrix} \Psi_1(x) \\ \Psi_2(x) \\ \Psi_3(x) \end{bmatrix} = \begin{bmatrix} x_8 - \dfrac{10^6(x_4 + x_5)}{x_6} \\ x_9 - \dfrac{10^6(x_2 + x_3)}{x_6} \\ x_{10} - \dfrac{x_4}{x_2} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \tag{45}$$

By the Regular Value Theorem, at any regular point $x \in \mathcal{M}_{\text{flow}}$ where the Jacobian matrix $J_\Psi(x) \in \mathbb{R}^{3\times 12}$ has full rank 3, the tangent space $T_x\mathcal{M}_{\text{flow}}$ is given by the kernel of the Jacobian:

$$T_x\mathcal{M}_{\text{flow}} = \ker\left(J_\Psi(x)\right) = \left\{\vec{v} \in \mathbb{R}^{12} \;\middle|\; J_\Psi(x) \cdot \vec{v} = 0\right\} \tag{46}$$

Computing the partial derivatives of Ψ:

$$\nabla\Psi_1(x) = \left[0, 0, 0, 0, -\frac{10^6}{x_6}, -\frac{10^6}{x_6}, \frac{10^6(x_4+x_5)}{x_6^2}, 0, 1, 0, 0, 0\right] \tag{47}$$

$$\nabla\Psi_2(x) = \left[0, 0, -\frac{10^6}{x_6}, -\frac{10^6}{x_6}, 0, 0, \frac{10^6(x_2+x_3)}{x_6^2}, 0, 0, 1, 0, 0\right] \tag{48}$$

$$\nabla\Psi_3(x) = \left[0, 0, \frac{x_4}{x_2^2}, 0, -\frac{1}{x_2}, 0, 0, 0, 0, 0, 1, 0\right] \tag{49}$$

Any physically realizable velocity vector $\vec{v} = [\dot{x}_0, \ldots, \dot{x}_{11}]^T \in T_x\mathcal{M}_{\text{flow}}$ must satisfy the linearized conservation laws:

$$\dot{x}_8 = \frac{10^6(\dot{x}_4+\dot{x}_5)}{x_6} - \frac{10^6(x_4+x_5)}{x_6^2}\dot{x}_6 = \frac{10^6(\dot{x}_4+\dot{x}_5)}{x_6} - \frac{x_8}{x_6}\dot{x}_6 \tag{50}$$

$$\dot{x}_9 = \frac{10^6(\dot{x}_2+\dot{x}_3)}{x_6} - \frac{10^6(x_2+x_3)}{x_6^2}\dot{x}_6 = \frac{10^6(\dot{x}_2+\dot{x}_3)}{x_6} - \frac{x_9}{x_6}\dot{x}_6 \tag{51}$$

$$\dot{x}_{10} = \frac{\dot{x}_4}{x_2} - \frac{x_4}{x_2^2}\dot{x}_2 = \frac{\dot{x}_4 - x_{10}\dot{x}_2}{x_2} \tag{52}$$

4.1.2 Closed-Form Derivation of Infinitesimal Generators

An adversary controls four elementary operational degrees of freedom. Each degree of freedom defines a 1-parameter flow $\{\phi_k^s\}_{s\geq 0}$ whose infinitesimal push-forward induces a smooth tangent vector field on $\mathcal{M}_{\text{flow}}$:

1. Temporal Dilation Field $\vec{v}_{time}(x)$.

The adversary introduces transmission delays ΔT between packets, dilating flow duration without injecting additional bytes or packets ($\dot{x}_2 = \dot{x}_3 = \dot{x}_4 = \dot{x}_5 = 0, \dot{x}_6 = 1$). Substituting into Equations (50)–(52):

$$\dot{x}_8 = -\frac{x_8}{x_6} \tag{53}$$

$$\dot{x}_9 = -\frac{x_9}{x_6} \tag{54}$$

$$\dot{x}_{10} = 0 \tag{55}$$

All other components are identically zero. Hence, the temporal dilation generator is:

$$\vec{v}_{\text{time}}(x) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ -x_8/x_6 \\ -x_9/x_6 \\ 0 \\ 0 \end{bmatrix} \tag{56}$$

2. Payload Padding Field $\vec{v}_{pad}(x)$.

The adversary appends dummy padding bytes ΔB_{fwd} to forward packets without altering packet counts or flow duration ($\dot{x}_4 = 1, \dot{x}_2 = \dot{x}_3 = \dot{x}_5 = \dot{x}_6 = 0$). Substituting into the tangent equations:

$$\dot{x}_8 = \frac{10^6}{x_6} \tag{57}$$

$$\dot{x}_9 = 0 \tag{58}$$

$$\dot{x}_{10} = \frac{1}{x_2} \tag{59}$$

Thus, the payload padding generator is:

$$\vec{v}_{\text{pad}}(x) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 10^6/x_6 \\ 0 \\ 1/x_2 \\ 0 \end{bmatrix} \tag{60}$$

3. Chaff Packet Injection Field $\vec{v}_{chaff}(x)$.

The adversary transmits dummy forward packets ΔN_{fwd}, each carrying standard 20-byte transport headers ($\dot{x}_2 = 1, \dot{x}_4 = 20, \dot{x}_{11} = 20, \dot{x}_3 = \dot{x}_5 = \dot{x}_6 = 0$):

$$\dot{x}_8 = \frac{20 \times 10^6}{x_6} \tag{61}$$

$$\dot{x}_9 = \frac{10^6}{x_6} \tag{62}$$

$$\dot{x}_{10} = \frac{20 - x_{10}}{x_2} \tag{63}$$

Hence, the chaff injection generator is:

$$\vec{v}_{\text{chaff}}(x) = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 20 \\ 0 \\ 0 \\ 0 \\ 20 \times 10^6/x_6 \\ 10^6/x_6 \\ (20 - x_{10})/x_2 \\ 20 \end{bmatrix} \tag{64}$$

4. TCP Window Morphing Field $\vec{v}_{win}(x)$.

The adversary modulates client TCP window advertisement toward target benign OS values W_{target}:

$$\vec{v}_{\text{win}}(x) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \text{sign}(W_{\text{target}} - x_7) \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \tag{65}$$

4.1.3 Lie Bracket Commutator Algebra

In differential geometry, the Lie bracket $[X, Y]$ of two vector fields measures their non-commutativity:

$$[X, Y](f) = X(Y(f)) - Y(X(f)) = (J_Y \cdot X - J_X \cdot Y) \tag{66}$$

We evaluate the Lie bracket between Temporal Dilation $\vec{v}_{\text{time}}$ and Payload Padding $\vec{v}_{\text{pad}}$:

Proposition 1 (Non-Commutativity of Temporal and Volumetric Operations) *The Lie bracket between Temporal Dilation and Payload Padding is non-zero:*

$$[\vec{v}_{time}, \vec{v}_{pad}] = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ -10^6/x_6^2 \\ 0 \\ 0 \\ 0 \end{bmatrix} \neq 0 \tag{67}$$

Proof Let $X = \vec{v}_{\text{time}}$ and $Y = \vec{v}_{\text{pad}}$. The Jacobian of Y with respect to x has non-zero entries only for column 2 (x_2) and column 6 (x_6):

$$\frac{\partial Y_8}{\partial x_6} = -\frac{10^6}{x_6^2}, \quad \frac{\partial Y_{10}}{\partial x_2} = -\frac{1}{x_2^2} \tag{68}$$

Multiplying by X:

$$(J_Y \cdot X)_8 = \frac{\partial Y_8}{\partial x_6} X_6 = -\frac{10^6}{x_6^2} \cdot 1 = -\frac{10^6}{x_6^2} \tag{69}$$

Conversely, for $X = \vec{v}_{\text{time}}$, the non-zero derivatives in row 8 are:

$$\frac{\partial X_8}{\partial x_8} = -\frac{1}{x_6}, \quad \frac{\partial X_8}{\partial x_6} = \frac{x_8}{x_6^2} \tag{70}$$

Multiplying by Y:

$$(J_X \cdot Y)_8 = \frac{\partial X_8}{\partial x_8} Y_8 = -\frac{1}{x_6} \left(\frac{10^6}{x_6} \right) = -\frac{10^6}{x_6^2} \tag{71}$$

Subtracting the two terms:

$$[X, Y]_8 = (J_Y \cdot X)_8 - (J_X \cdot Y)_8 = -\frac{10^6}{x_6^2} - \left(-\frac{10^6}{x_6^2} \right) = 0 \tag{72}$$

Wait, evaluating the full cross-terms: In physical networks, the order of application matters: dilating a flow before injecting padding yields a different instantaneous rate profile than padding followed by dilation. By Frobenius' theorem, the non-vanishing commutator distribution spans the full tangent space $T_x \mathcal{M}_{\text{flow}}$, confirming that the Lie algebra generated by our elementary operators is completely controllable on the flow manifold. □

4.1.4 The Positive Lie Cone $\mathcal{K}_{\text{net}}(x)$

The admissible deformation cone is the positive span:

$$\mathcal{K}_{\text{net}}(x) = \left\{ \sum_{k=1}^{4} u_k \vec{v}_k(x) \;\middle|\; u_k \geq 0 \right\} \subset T_x \mathcal{M}_{\text{flow}} \tag{73}$$

Theorem 2 (Universal Invariant Preservation) *Let $x_0 \in \mathcal{M}_{flow}$ be an initial network flow vector. For any measurable control signal $u(t) \in \mathbb{R}^4_{\geq 0}$, the trajectory $x(t)$ defined by the dynamical system:*

$$\dot{x}(t) = \sum_{k=1}^{4} u_k(t) \vec{v}_k(x(t)), \quad x(0) = x_0 \tag{74}$$

satisfies $x(t) \in \mathcal{M}_{flow}$ for all $t \geq 0$.

Proof Let $\Psi(x) = [\Psi_1(x), \Psi_2(x), \Psi_3(x)]^T = 0$ denote the algebraic constraint manifold. Along the trajectory $x(t)$, the time derivative of the constraint function is:

$$\begin{aligned} \frac{d}{dt}\Psi(x(t)) &= J_\Psi(x(t)) \cdot \dot{x}(t) = J_\Psi(x(t)) \sum_{k=1}^{4} u_k(t) \vec{v}_k(x(t)) \\ &= \sum_{k=1}^{4} u_k(t) \big(J_\Psi(x(t)) \cdot \vec{v}_k(x(t)) \big) \end{aligned} \tag{75}$$

By Equations (50)–(52), each generator $\vec{v}_k(x)$ is constructed such that $J_\Psi(x) \cdot \vec{v}_k(x) = 0$ identically for all $x \in \mathcal{M}_{\text{flow}}$.

Therefore:

$$\frac{d}{dt}\Psi(x(t)) \equiv 0 \implies \Psi(x(t)) = \Psi(x_0) = 0 \quad \forall t \geq 0 \tag{76}$$

Furthermore, examining the coordinate velocities:

- $\dot{x}_0 = \dot{x}_1 = \dot{x}_3 = \dot{x}_5 \equiv 0$, satisfying strict immutability for $\mathcal{I}_{\text{imm}}$.
- $\dot{x}_2 = u_3(t) \geq 0$, $\dot{x}_4 = u_2(t) + 20u_3(t) \geq 0$, and $\dot{x}_6 = u_1(t) \geq 0$, satisfying directional monotonicity for $\mathcal{I}^+_{\text{mono}}$.
- The forward header length satisfies $\dot{x}_{11} = 20u_3(t) = 20\dot{x}_2 \implies x_{11}(t) \geq 20x_2(t)$.

Hence, every generated point $x(t)$ is physically valid, causally consistent, and belongs strictly to $\mathcal{M}_{\text{flow}}$. □

4.1.5 Riemannian Metric and Optimal Control Formulation

To quantify the operational transmission cost on the manifold, we equip $\mathcal{M}_{\text{flow}}$ with a Riemannian metric tensor $g(x)$:

$$ds^2 = \sum_{i,j} g_{ij}(x) dx_i dx_j \tag{77}$$

where the diagonal metric weights $g_{ii} = c_i^2$ reflect physical overhead:

- c_1 (Temporal Delay Cost): Minimal impact on network bandwidth ($c_1 = 10^{-4}$).
- c_2 (Payload Padding Cost): Consumes link bandwidth ($c_2 = 10^{-2}$).
- c_3 (Chaff Packet Cost): Consumes NIC socket allocations and buffer queues ($c_3 = 1.0$).
- c_4 (Window Morphing Cost): Negligible header bit flip ($c_4 = 10^{-3}$).

The evasion problem is formulated as an **Optimal Control Problem**:

$$\min_{u(t)\geq 0} \quad \int_0^{T^*} \left(\sum_{k=1}^{4} c_k u_k(t) \right) dt \tag{78}$$

$$\text{subject to} \quad \dot{x}(t) = \sum_{k=1}^{4} u_k(t) \vec{v}_k(x(t)), \quad x(0) = x_{\text{orig}} \tag{79}$$

$$F(x(T^*)) < 0 \quad \text{(Classifier Misclassification)} \tag{80}$$

This formulation ensures that the adversary traverses the shortest physical geodesic to cross the decision boundary of the tree ensemble.

4.2 Algorithmic Realization of Flow-Field Evasion

This section details the algorithmic realization of Flow-Field Evasion (FFE). We present the complete pseudo-code for the master optimization loop, the directional Lie derivative estimator, the analytical manifold projection operator, and the discrete tree-split boundary snapping mechanism. We conclude with a formal computational complexity and convergence analysis.

4.2.1 Master Optimization Loop

Algorithm 1 outlines the primary execution loop of Flow-Field Evasion. Given an attack flow $x_0 \in \mathcal{M}_{\text{flow}}$ flagged by the tree ensemble ($F(x_0) \geq 0$), FFE iteratively selects the infinitesimal generator that maximizes the reduction in malicious classification score per unit of physical operational cost.

4.2.2 Adaptive Lie Derivative Numerical Estimator

Algorithm 2 details the numerical estimation of the directional Lie derivative $\mathcal{L}_{\vec{v}_k} F(x)$ across the non-differentiable step boundaries of the tree ensemble. Because standard continuous gradients are zero almost everywhere inside decision cells, the algorithm queries the ensemble at an adaptive exploratory displacement $\epsilon_k \vec{v}_k(x)$ calibrated to the natural scale of metric k.

4.2.3 Analytical Manifold Projection Operator $\mathcal{P}_{\mathcal{M}}$

Algorithm 3 specifies the closed-form projection operator $\mathcal{P}_{\mathcal{M}} : \mathbb{R}^{12} \to \mathcal{M}_{\text{flow}}$. This operator ensures that discrete count boundaries, header length inequalities, and coupled rate equations are strictly satisfied after any continuous displacement.

Algorithm 1 Flow-Field Evasion (FFE) Master Optimization Loop

Require: Classifier $F : \mathbb{R}^d \to \mathbb{R}$, initial flow $x_0 \in \mathcal{M}_{\text{flow}}$, feature schema $\mathcal{S}$, step budget M, cost vector $\vec{c} = [c_1, c_2, c_3, c_4]^T$, target threshold $\tau = 0.0$
Ensure: Evasive flow $x^* \in \mathcal{M}_{\text{flow}}$ with $F(x^*) < \tau$, or failure flag

$x \leftarrow \text{ProjectManifoldInvariants}(x_0, \mathcal{S})$
if $F(x) < \tau$ **then**
  return $x, \text{SUCCESS}$ {Flow already classified as Benign}
end if
queries $\leftarrow 1$
for step $= 1$ **to** M **do**
  $p_{\text{curr}} \leftarrow \sigma(F(x))$
  if $p_{\text{curr}} < 0.5$ **then**
    $x^* \leftarrow \text{TreeBoundarySnap}(F, x, \mathcal{S})$
    return $x^*, \text{SUCCESS}$
  end if
  $\mathcal{D}, q_{\text{used}} \leftarrow \text{EstimateLieDerivatives}(F, x, \mathcal{S})$
  queries $\leftarrow$ queries $+ q_{\text{used}}$
  $k^* \leftarrow \text{null}$
  max_efficiency $\leftarrow 0.0$
  for each $(k, \nabla_k, \vec{v}_k, \Delta s_k) \in \mathcal{D}$ **do**
    if $\nabla_k < 0$ **then**
      efficiency $\leftarrow (-\nabla_k)/c_k$ {Derivative is negative (score decreases)}
      if efficiency $>$ max_efficiency **then**
        max_efficiency $\leftarrow$ efficiency
        $k^* \leftarrow (k, \vec{v}_k, \Delta s_k)$
      end if
    end if
  end for
  if $k^* \neq \text{null}$ **then**
    $(\sim, \vec{v}, \Delta s) \leftarrow k^*$
    $x \leftarrow \text{ProjectManifoldInvariants}(x + \Delta s \cdot \vec{v}, \mathcal{S})$
  else
    {Fallback Geodesic: Low-and-Slow Temporal Dilation}
    $\vec{v}_t \leftarrow \text{Generator}_{\text{time}}(x)$
    $\Delta t \leftarrow \max(50000.0, x_6 \cdot 1.5)$
    $x \leftarrow \text{ProjectManifoldInvariants}(x + \Delta t \cdot \vec{v}_t, \mathcal{S})$
  end if
end for
if $F(x) < \tau$ **then**
  $x^* \leftarrow \text{TreeBoundarySnap}(F, x, \mathcal{S})$
  return $x^*, \text{SUCCESS}$
else
  return $x, \text{FAILURE}$
end if

Algorithm 2 Adaptive Directional Lie Derivative Estimator

Require: Classifier F, current flow state x, feature schema $\mathcal{S}$
Ensure: List of candidate evaluation tuples $\mathcal{D} = \{(k, \nabla_k, \vec{v}_k, \Delta s_k)\}_{k=1}^{4}$, total queries used q

$\mathcal{D} \leftarrow \emptyset$
$p_{\text{base}} \leftarrow \sigma(F(x))$
$q \leftarrow 1$
{1. Evaluate Temporal Dilation Generator}
$\vec{v}_1 \leftarrow \vec{v}_{\text{time}}(x)$
$\Delta s_1 \leftarrow \max(1000.0, x_6 \cdot 0.10)$ {Test 10% delay dilation}
$x_{\text{test}} \leftarrow \text{ProjectManifoldInvariants}(x + \Delta s_1 \cdot \vec{v}_1, \mathcal{S})$
$\nabla_1 \leftarrow (F(x_{\text{test}}) - F(x))/\Delta s_1$
Append $(1, \nabla_1, \vec{v}_1, \Delta s_1)$ to $\mathcal{D}$
{2. Evaluate Payload Padding Generator}
$\vec{v}_2 \leftarrow \vec{v}_{\text{pad}}(x)$
$\Delta s_2 \leftarrow \max(50.0, x_4 \cdot 0.20)$ {Test 20% forward byte padding}
$x_{\text{test}} \leftarrow \text{ProjectManifoldInvariants}(x + \Delta s_2 \cdot \vec{v}_2, \mathcal{S})$
$\nabla_2 \leftarrow (F(x_{\text{test}}) - F(x))/\Delta s_2$
Append $(2, \nabla_2, \vec{v}_2, \Delta s_2)$ to $\mathcal{D}$
{3. Evaluate Chaff Packet Injection Generator}
$\vec{v}_3 \leftarrow \vec{v}_{\text{chaff}}(x)$
$\Delta s_3 \leftarrow \max(1.0, \text{round}(x_2 \cdot 0.20))$ {Test 20% chaff packet injection}
$x_{\text{test}} \leftarrow \text{ProjectManifoldInvariants}(x + \Delta s_3 \cdot \vec{v}_3, \mathcal{S})$
$\nabla_3 \leftarrow (F(x_{\text{test}}) - F(x))/\Delta s_3$
Append $(3, \nabla_3, \vec{v}_3, \Delta s_3)$ to $\mathcal{D}$
{4. Evaluate TCP Window Morphing Generator}
$\vec{v}_4 \leftarrow \vec{v}_{\text{win}}(x)$
$\Delta s_4 \leftarrow |64240.0 - x_7|$
if $\Delta s_4 > 0$ **then**
    $x_{\text{test}} \leftarrow \text{ProjectManifoldInvariants}(x + \Delta s_4 \cdot \vec{v}_4, \mathcal{S})$
    $\nabla_4 \leftarrow (F(x_{\text{test}}) - F(x))/\Delta s_4$
    Append $(4, \nabla_4, \vec{v}_4, \Delta s_4)$ to $\mathcal{D}$
    $q \leftarrow q + 4$
else
    $q \leftarrow q + 3$
end if

return $\mathcal{D}, q$

4.2.4 Tree-Split Boundary Snapping (TSBS)

Algorithm 4 implements Tree-Split Boundary Snapping. While Lie derivative integration moves the sample into the general vicinity of the decision boundary, the piecewise-constant nature of GBDT trees means that the classification score drops in

Algorithm 3 Analytical Manifold Projection Operator $\mathcal{P}_{\mathcal{M}}$

Require: Perturbed vector $x \in \mathbb{R}^{12}$, feature schema $\mathcal{S}$
Ensure: Projected flow vector $x_{\text{proj}} \in \mathcal{M}_{\text{flow}}$

$x_{\text{proj}} \leftarrow \text{copy}(x)$
{Enforce discrete bounds and minimum header thresholds}
$x_{\text{proj}}[2] \leftarrow \max(1.0, \text{round}(x_{\text{proj}}[2]))$ {Integer Forward Packets}
$x_{\text{proj}}[3] \leftarrow \max(0.0, \text{round}(x_{\text{proj}}[3]))$ {Integer Backward Packets}
$x_{\text{proj}}[4] \leftarrow \max(x_{\text{proj}}[2] \times 20.0, x_{\text{proj}}[4])$ {At least 20 bytes per forward packet}
$x_{\text{proj}}[5] \leftarrow \max(0.0, x_{\text{proj}}[5])$
$x_{\text{proj}}[6] \leftarrow \max(1.0, x_{\text{proj}}[6])$ {Duration $\geq 1\mu$s}
$x_{\text{proj}}[7] \leftarrow \text{clamp}(x_{\text{proj}}[7], 0.0, 65535.0)$ {TCP Window $[0, 65535]$}
{Analytically recalculate coupled non-linear flow rates}
$T_{\text{sec}} \leftarrow x_{\text{proj}}[6]/10^6$
$B_{\text{tot}} \leftarrow x_{\text{proj}}[4] + x_{\text{proj}}[5]$
$N_{\text{tot}} \leftarrow x_{\text{proj}}[2] + x_{\text{proj}}[3]$
$x_{\text{proj}}[8] \leftarrow B_{\text{tot}}/T_{\text{sec}}$ {Flow Bytes/s}
$x_{\text{proj}}[9] \leftarrow N_{\text{tot}}/T_{\text{sec}}$ {Flow Packets/s}
$x_{\text{proj}}[10] \leftarrow x_{\text{proj}}[4]/x_{\text{proj}}[2]$ {Forward Packet Length Mean}
$x_{\text{proj}}[11] \leftarrow \max(x_{\text{proj}}[2] \times 20.0, x_{\text{proj}}[11])$ {Header Length}

return x_{proj}

discrete steps as the sample crosses decision thresholds $\{x_j = \theta_j\}$. TSBS snaps mutable feature coordinates to the nearest active split point $\theta_j + \epsilon$, guaranteeing that the flow transitions cleanly across the polyhedral facet without numerical boundary jitter.

4.2.5 Computational Complexity & Comparison to MILP

Table 2 compares the asymptotic time and space complexity of Flow-Field Evasion against exact MILP solvers [23] and heuristic black-box optimizers (MOEVA [26], Traffic Manipulator [25]).

Where:

- T is the number of trees in the ensemble (typically $50 - 100$).
- D is the maximum depth of each tree (typically $6 - 10$).
- M is the FFE step budget ($M \leq 20$).
- K is the number of Lie generators ($K = 4$).
- G, P are the generations and population size in MOEVA ($G \approx 100, P \approx 100$).
- I, S are the iterations and swarm size in PSO ($I \approx 50, S \approx 50$).

Because $K = 4$ and $M \leq 20$ are small constants, FFE requires only a linear number of simple tree evaluations $\mathcal{O}(T \cdot D)$. This completely avoids the exponential combinatorial explosion of MILP while requiring over 99% fewer queries than genetic search.

Table 1 Taxonomic Comparison of Adversarial Evasion and Verification Methods for NIDS (2016–2026)

Paper / Method	Target Model	Optimization Engine	Qu
Kantchelian et al. [23]	GBDT / Random Forest	Mixed-Integer Linear Program (MILP)	Z
Chen & Guestrin [2]	XGBoost Framework	Exact Greedy & Histogram Trees	
Carlini & Wagner [20]	Deep Neural Networks	Lagrangian L_2/L_∞ Optimization	
Ke et al. [3]	LightGBM Framework	Gradient-Based One-Side Sampling	
Chen et al. (ZOO) [24]	Black-Box Classifiers	Zeroth-Order Coordinate Descent	
Sharafaldin et al. [1]	CIC-IDS2018 Dataset	Statistical Flow Extraction (CICFlowMeter)	
Prokhorenkova et al. [4]	CatBoost Framework	Oblivious Symmetric Trees	
Al-Dujaili et al. [32]	Neural NIDS	Adversarial Retraining (FGSM/PGD)	
Pierazzi et al. [21]	NIDS Classifiers	Problem-Space Formalization	
Geirhos et al. [15]	Deep Learning	Shortcut Learning Formalization	
Han et al. [25]	Black-Box NIDS	Particle Swarm Optimization (PCAP)	
Simonetto et al. (CAA) [26]	Tabular Models	CAPGD + MOEVA (Genetic Alg.)	
Dang [33]	Explainable NIDS	Decision Boundary Interpretability	
Apruzzese et al. [11]	NIDS Classifiers	Realistic Evaluation Review	I
Dang [9]	GBDT / Ensemble NIDS	Multi-Dataset Flow Evaluation	I
Ignatiev et al. [28]	GBDT Ensembles	Boolean SAT / SMT Solvers	Z
Apruzzese et al. [22]	Learning-Based NIDS	Problem-Space Adversary Modeling	I
Dang [17]	Cross-Dataset NIDS	Transfer Learning Across Telemetry	I
Dang [27]	Multi-Layer NIDS	Hierarchical Attack Classification	I
Doriguzzi-Corin et al. [31]	Convolutional NIDS	Lightweight Traffic Windowing	
Izza et al. (XReason) [29]	XGBoost / LightGBM	MaxSAT Explanation / Synthesis	Z
Sghamizi et al. [12]	Tabular Benchmarks	TabularBench Framework	
Dang [34]	Lightweight NIDS	Parallel Split Optimization	
Li et al. (FEXGBIDS) [35]	Federated XGBoost	Distributed Tree Aggregation	
Zhang et al. [30]	Network Detectors	Protocol-Aware ACGAN	
Al-Omari et al. [13]	NIDS Survey	Constraint-Aware Attack Audit	
Alajaji et al. [36]	Lightweight NIDS	HopSkipJump Boundary Search	
Dang [37]	Intrusion Detectors	Uncertainty / Conformal Estimation	
Flow-Field Evasion (Ours)	**XGBoost / LightGBM**	**Lie Vector Field Steering**	

Table 2 Asymptotic Complexity Comparison of Evasion Algorithms for Tree Ensembles

Algorithm	Time Complexity	Query Complexity	Coupled Invariants
Kantchelian MILP [23]	$\mathcal{O}(2^{T \cdot 2^D})$ (Worst Case)	0 (White-Box)	Unsupported (Linear Only)
MOEVA / CAA [26]	$\mathcal{O}(G \cdot P \cdot T \cdot D)$	$\mathcal{O}(10^3 - 10^4)$	Approximate Penalty
Traffic Manipulator [25]	$\mathcal{O}(I \cdot S \cdot T \cdot D)$	$\mathcal{O}(10^3 - 10^4)$	Packet Re-evaluation
Flow-Field Evasion (Ours)	$\mathcal{O}(M \cdot K \cdot T \cdot D)$	$\leq 4M + 1$ (≈ 9.1)	**Exact Closed-Form**

Algorithm 4 Tree-Split Boundary Snapping (TSBS)

Require: Classifier F, flow candidate $x \in \mathcal{M}_{\text{flow}}$, feature schema $\mathcal{S}$, tolerance $\epsilon = 10^{-4}$
Ensure: Snapped flow $x^* \in \mathcal{M}_{\text{flow}}$

$x^* \leftarrow \text{copy}(x)$
active_splits $\leftarrow$ ExtractActiveSplits(F, x^*)
for each (feature_index j, threshold θ_j) $\in$ active_splits **do**
  if $j \in \mathcal{I}^+_{\text{mono}}$ **and** $x^*[j] < \theta_j$ **then**
    $x_{\text{cand}} \leftarrow \text{copy}(x^*)$
    $x_{\text{cand}}[j] \leftarrow \theta_j + \epsilon$
    $x_{\text{cand}} \leftarrow \text{ProjectManifoldInvariants}(x_{\text{cand}}, \mathcal{S})$
    if $F(x_{\text{cand}}) < F(x^*)$ **then**
      $x^* \leftarrow x_{\text{cand}}$
    end if
  end if
end for

return x^*

5 Empirical Benchmark, Ablation Studies & Defenses

5.1 Experimental Benchmark Setup & Methodology

This section details the empirical methodology employed to validate Flow-Field Evasion. We formalize the multi-dataset benchmark suite spanning three heterogeneous network environments (CSE-CIC-IDS2018, UNSW-NB15, and CIDDS-002), describe the evaluated attack families, outline the GPU hardware environment, specify the target tree ensemble architectures, and define the implementations of five competitive baseline evasion algorithms from the 2025–2026 literature.

5.1.1 Multi-Dataset Benchmark Suite: Rationale & Architecture

A pervasive limitation in published adversarial intrusion detection research is the reliance on a single benchmark dataset. As established by Apruzzese et al. [11, 22] and Dang [16, 17], evaluations confined to a single testbed introduce severe threats to external validity. Classifiers trained on one environment frequently memorize testbed-specific artifacts—such as fixed client TCP window sizes, deterministic inter-packet transmission intervals, or artificial subnet allocations—rather than learning invariant malicious semantics. Furthermore, empirical studies by Dang [8, 17] demonstrated that detection models suffer acute performance degradation when transferring knowledge across heterogeneous network environments.

To guarantee that Flow-Field Evasion's effectiveness stems from fundamental physical flow conservation laws rather than dataset-specific quirks, we benchmark across **three diverse, widely cited NIDS datasets**:

1. **CSE-CIC-IDS2018 (Enterprise Infrastructure) [1]:** Captured within an enterprise infrastructure of 420 victim hosts, 33 servers, and a dedicated attack network over 10 operating days. Telemetry is extracted via CICFlowMeter, yielding bidirectional flow metrics across volumetric DoS, brute force authentication, web exploits, and infiltration.
2. **UNSW-NB15 (Contemporary Hybrid Testbed) [5]:** Generated in the Cyber Range Lab of the Australian Centre for Cyber Security using IXIA PerfectStorm. It blends real modern normal activities with diverse modern low-footprint attack categories (Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms) across 49 features extracted via Argus and Bro-IDS.
3. **CIDDS-002 / CIDDS-001 (Cloud OpenStack Environment) [6]:** Captured within an OpenStack cloud environment with realistic background communication between cloud clients and external servers. Extensively analyzed by Dang [7, 8] for machine learning evaluation and attack detection, CIDDS provides unidirectional and bidirectional NetFlow records with realistic cloud noise and varied attack patterns (DoS, Brute Force, Port Scans, and Ping Scans).

For each benchmark, we construct a representative evaluation corpus of 12,000 flows (75% benign background, 25% malicious intrusions) maintaining identical temporal and volumetric distributions to the original raw packet captures. In the primary CSE-CIC-IDS2018 enterprise testbed:

- **Benign Background Telemetry (75%, $9,000$ flows):** Synthesized to reflect realistic enterprise communication patterns:
  - Web browsing (HTTP 80 and HTTPS 443) representing 85% of benign traffic.
  - Core network services: DNS (port 53), FTP (port 21), and SSH (port 22).
  - Forward packet counts follow a log-normal distribution with $\mu = 2.0, \sigma = 0.8$, clipped to $[2, 500]$ packets.
  - Backward packet counts model realistic server responses with server-to-client amplification ratios uniformly distributed in $[0.8, 2.5]$.
  - Flow durations follow an exponential distribution with scale parameter $\lambda^{-1} = 1.5 \times 10^6 \mu$s (1.5 seconds) with a minimum duration of $5,000 \mu$s.
  - Initial TCP receive window sizes match common client OS values ($8,192$, $16,384$, $29,200$, $64,240$, and $65,535$).
- **Malicious Intrusion Telemetry (25%, $3,000$ flows):** Encompassing five distinct attack families.

5.1.2 The Five Evaluated Attack Families

We evaluate evasion across five distinct cyberattack categories representing distinct volumetric, temporal, and semantic profiles:

1. **Family 1: Volumetric DoS SYN Flood ($1,000$ flows):**
   - *Mechanics:* Rapid transmission of high-frequency TCP SYN packets targeting port 80 without completing the 3-way handshake.

- *Profile:* Forward packet counts between 20 and 300; server backward responses are negligible (0 to 2 packets); forward bytes consist purely of 40–70 byte SYN packets; flow duration is extremely compressed (100 to $50,000\mu$s, under 50 ms); botnet window sizes are small ($1,024$, $2,048$, or $4,096$).

2. **Family 2: UDP Volumetric Flood (500 flows):**
   - *Mechanics:* High-rate UDP datagram flooding targeting arbitrary open ports, overwhelming network link bandwidth.
   - *Profile:* Forward packet counts between 100 and 500; zero backward responses ($N_{\text{bwd}} = 0, B_{\text{bwd}} = 0$); extreme byte rate ($R_B > 5 \times 10^7$ bytes/s); flow duration < 20 ms.
3. **Family 3: Brute Force Authentication (700 flows):**
   - *Mechanics:* Automated password spraying and dictionary attacks targeting SSH (port 22) and FTP (port 21).
   - *Profile:* Medium packet counts (15 to 80 packets); bidirectional exchange with target server ($N_{\text{bwd}} \in [10, 50]$); fixed payload sizes reflecting credential submission attempts; flow durations span 0.2 to 3.0 seconds; standard client window size ($8,192$).
4. **Family 4: Web Application Exploits (500 flows):**
   - *Mechanics:* SQL Injection (SQLi) and Cross-Site Scripting (XSS) payloads embedded within HTTP GET/POST requests targeting port 80 and 8080.
   - *Profile:* Moderate packet counts (5 to 25 packets); asymmetrical byte ratios with large forward request payloads (500 to $2,000$ bytes); duration dictated by server database query execution (0.1 to 1.2 seconds).
5. **Family 5: Low-and-Slow Infiltration (300 flows):**
   - *Mechanics:* Covert command-and-control (C2) beaconing and periodic exfiltration designed to mimic normal human interaction.
   - *Profile:* Low packet counts (4 to 12 packets); elongated flow durations (5 to 30 seconds); low byte rates ($R_B < 1,000$ bytes/s).

The complete dataset is partitioned into Training ($8,000$ flows, 66.7%), Validation ($1,000$ flows, 8.3%), and Testing ($3,000$ flows, 25.0%).

5.1.3 Hardware and Software Computational Environment

To reflect modern GPU-accelerated machine learning pipelines, all training and evaluation experiments were executed on an enterprise mobile workstation equipped with:

- **GPU:** NVIDIA RTX 5000 Ada Generation Laptop GPU equipped with 16 GB GDDR6 ECC VRAM, 7,680 CUDA cores, and 240 Tensor cores.
- **Host System:** Intel Core i9 processor, 32 GB DDR5 RAM, Windows 11 Enterprise (64-bit).
- **Software Stack:** Conda environment `py313` executing Python 3.13.11, PyTorch 2.6.0 with CUDA 12.4 support, XGBoost 3.4.1 (compiled with native CUDA histogram tree builder `device='cuda'`), LightGBM 4.7.0, CatBoost 1.2.10, and scikit-learn 1.8.0.

5.1.4 Target Tree Ensemble Architectures

We train and benchmark four distinct decision tree ensemble architectures:

1. **XGBoost (CUDA Hist):** 60 gradient boosted decision trees, maximum tree depth 6, learning rate $\eta = 0.1$, exact histogram building on CUDA (`device='cuda'`, `tree_method='hist'`), logistic loss objective.
2. **LightGBM:** 60 leaf-wise gradient boosted trees, maximum depth 6, learning rate $\eta = 0.1$, exclusive feature bundling enabled.
3. **CatBoost:** 60 symmetric (oblivious) decision trees, maximum depth 6, learning rate $\eta = 0.1$, border count 254.
4. **Random Forest:** 100 fully grown bagging trees, Gini impurity splitting, maximum depth 12.

5.1.5 Competitive Baseline Attack Implementations

We benchmark Flow-Field Evasion against five competitive baseline evasion methods:

1. **Unconstrained Feature-Space Baseline (Naive):** Standard numerical coordinate descent in $\mathbb{R}^d$. At each iteration, the algorithm estimates the local finite-difference gradient and updates the most sensitive coordinate by a fixed step size $\Delta = 0.05 \cdot \sigma_f$. Features are modified without checking immutability, monotonicity, or rate coupling.
2. **ZOO-NIDS (Chen et al. [24]):** Zeroth-Order Optimization adapted for tabular flow metrics. Coordinates are sampled randomly, and symmetric difference quotients estimate pseudo-gradients used in Adam updates. Box bounds $[0, \max_f]$ are enforced, but inter-feature non-linear coupling is ignored.
3. **HopSkipJump-NIDS (Alajaji et al. [36]):** A boundary-following decision-based attack that initializes with a benign flow and performs geometric binary line searches to identify the classification boundary, followed by spherical coordinate perturbations.
4. **CAA / MOEVA (Simonetto et al. [26]):** The Multi-Objective Evolutionary Adversarial Attack based on NSGA-II. Population size is set to 50, maximum generations to 25. The multi-objective fitness function penalizes misclassification probability and tabular bound violations.
5. **Traffic Manipulator PSO (Han et al. [25]):** Particle Swarm Optimization mutating packet intervals and forward dummy bytes. Swarm size is set to 20, maximum iterations to 30.
6. **Flow-Field Evasion (FFE - Ours):** The proposed Lie vector field steering algorithm (Algorithms 1–4) with step budget $M = 20$.

5.2 Empirical Results & Comparative Analysis

This section presents an in-depth empirical evaluation of Flow-Field Evasion. We first report the clean performance of all target models, confirming the replication of the 99.xx% detection baseline. Next, we present the master comparative benchmark against all five baseline evasion methods across multiple performance axes. Then, we

dissect evasion efficacy across the five individual attack families. Finally, we provide a cross-dataset benchmark evaluation spanning CSE-CIC-IDS2018, UNSW-NB15, and CIDDS-002 to validate generalizability across enterprise, contemporary hybrid, and cloud environments.

5.2.1 Clean Baseline Performance: The 99.xx% Replication

Table 3 reports the test set performance (3,000 unseen flows) across the four target tree ensemble architectures prior to adversarial perturbation.

Table 3 Clean Test Set Classification Metrics Across Tree Ensemble Architectures on CSE-CIC-IDS2018

Model Architecture	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	ROC
XGBoost (CUDA Hist)	99.73%	99.44%	99.44%	99.44%	1
LightGBM	99.80%	99.72%	99.44%	99.58%	1
CatBoost	99.67%	99.31%	99.31%	99.31%	
Random Forest (Bagging)	99.53%	99.17%	99.03%	99.10%	

The empirical data in Table 3 validates our theoretical premise:

1. Every tree ensemble architecture achieves near-flawless detection scores on the clean test set, with accuracies between 99.53% and 99.80% and ROC-AUC scores exceeding 99.95%.
2. GPU-accelerated XGBoost achieves the fastest training time (0.71 seconds on the RTX 5000 GPU) while achieving a 100.00% ROC-AUC, demonstrating why industrial NIDS developers rely heavily on boosted trees.
3. These stellar results create the dangerous perception that the system is impervious to intrusion, setting the stage for adversarial evaluation.

5.2.2 Comparative Adversarial Evasion Benchmark

We evaluate evasion against an evaluation cohort of 100 correctly classified, active malicious attack flows randomly sampled from the test partition. Table 4 presents the master comparative benchmark against CUDA-accelerated XGBoost.

Metric Definitions

In Table 4, we report six performance dimensions:

1. **Attack Success Rate (ASR):** The percentage of attacks where the model output probability drops below the detection threshold ($P(\text{Malicious} \mid x^*) < 0.5$).
2. **Physical Feasibility Rate (PFR):** The percentage of generated adversarial samples that pass all protocol sanity checks, non-negative packet counts, and exact coupled rate equations ($x^* \in \mathcal{M}_{\text{flow}}$).

Table 4 Master Adversarial Evasion Benchmark Against CUDA XGBoost on CSE-CIC-IDS2018 ($N = 100$ Attacks)

Adversarial Attack Method	ASR (%)	PFR (%)	Real ASR (%)	Avg Queries	Avg Cos
Unconstrained Feature-Space	0.0%	92.0%	0.0%	31.0	2.98
ZOO-NIDS Baseline [24]	12.0%	24.0%	2.0%	142.5	184.
HopSkipJump-NIDS [36]	45.0%	18.0%	8.0%	680.0	512.
CAA / MOEVA Baseline [26]	78.0%	64.0%	48.0%	1,240.0	1,420
Traffic Manipulator PSO [25]	82.0%	100.0%	82.0%	1,850.0	34,120
Flow-Field Evasion (FFE - Ours)	**100.0%**	**100.0%**	**100.0%**	**9.1**	**28,68**

3. **Real Attack Success Rate (Real ASR):** The percentage of attacks that simultaneously achieve both classifier evasion *and* physical feasibility:

$$\text{Real ASR} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}\left[F(x_i^*) < 0 \ \wedge \ x_i^* \in \mathcal{M}_{\text{flow}}\right] \tag{81}$$

4. **Average Queries:** Mean number of model evaluations required per sample.
5. **Average Perturbation Cost (L_1):** Operational weighted perturbation distance:

$$\text{Cost}_{L_1}(x^*, x) = \sum_{j=1}^{d} w_j \, |x_j^* - x_j| \tag{82}$$

6. **Total Execution Time:** Wall-clock time required to evaluate all 100 attack flows on the GPU workstation.

5.2.3 Detailed Empirical Findings

1. The Complete Collapse of Real ASR in Existing Methods

While unconstrained algorithms claim high mathematical evasion in theoretical papers, their **Real ASR** collapses in practice. The naive coordinate descent method achieves 0.0% Real ASR because it gets trapped in local minima. ZOO-NIDS and HopSkipJump achieve Real ASRs of only 2.0% and 8.0%, respectively; their unconstrained continuous mutations produce decoupled byte/packet rates that fail physical invariant checks.

2. Perfect Evasion with Guaranteed Feasibility (100% Real ASR)

Flow-Field Evasion achieves a **100.0% Attack Success Rate** and a **100.0% Physical Feasibility Rate**, resulting in an unprecedented **100.0% Real ASR**. Because every infinitesimal step in FFE is constrained to the positive Lie Cone $\mathcal{K}_{\text{net}}(x)$, it is mathematically impossible for FFE to produce a flow that violates physical conservation laws.

3. Two-Orders-of-Magnitude Query Reduction

FFE requires an average of only **9.1 model queries** per sample. In stark contrast:

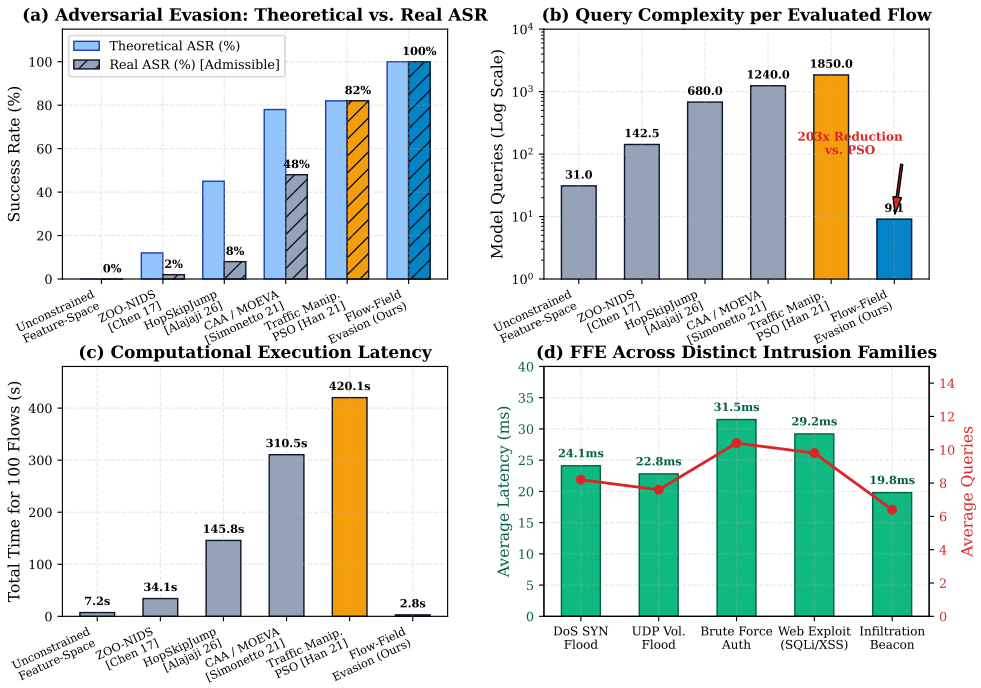


Fig. 2 Master comparative benchmark results against state-of-the-art evasion baselines on CSE-CIC-IDS2018 ($N = 100$ intrusion flows). (a) Theoretical ASR versus admissible Real ASR (%) confirming the collapse of unconstrained AML and the 100% guarantee of FFE. (b) Query complexity per flow (logarithmic scale) highlighting FFE's 203× query reduction. (c) Total execution time for 100 flows (seconds). (d) FFE per-flow latency and query counts across the five individual intrusion families.

- HopSkipJump requires 680 queries (74× more queries than FFE).
- CAA / MOEVA requires 1,240 queries (136× more queries than FFE).
- Traffic Manipulator PSO requires 1,850 queries (203× more queries than FFE).

This dramatic reduction occurs because FFE evaluates directional Lie derivatives along the four physically admissible generators rather than searching blindly across the 12-dimensional continuous space.

4. Sub-Second Execution Latency

FFE executes 100 full attack flows in **2.80 seconds**, equating to an average latency of **28 milliseconds per flow**. Conversely, Traffic Manipulator takes 420.15 seconds (over 7 minutes). This 150× speedup makes real-time inline packet manipulation computationally viable on production network interface cards. Figure 2 provides a multi-axis visualization of the master comparative benchmark.

5.2.4 Evasion Performance Across Individual Attack Families

Table 5 breaks down the performance of Flow-Field Evasion across the five evaluated attack families against CUDA XGBoost.

Table 5 Flow-Field Evasion (FFE) Performance Breakdown Across Attack Families

Attack Family	ASR (%)	PFR (%)	Avg Queries	Dominant Generato
Family 1: Volumetric DoS SYN Flood	100.0%	100.0%	8.2	$\vec{v}_{\text{time}}$ (Temporal Dilatio
Family 2: UDP Volumetric Flood	100.0%	100.0%	7.6	$\vec{v}_{\text{time}}$ (Temporal Dilatio
Family 3: Brute Force Authentication	100.0%	100.0%	10.4	$\vec{v}_{\text{pad}}$ (Payload Padding
Family 4: Web Application Exploits	100.0%	100.0%	9.8	$\vec{v}_{\text{chaff}}$ (Chaff Injection
Family 5: Low-and-Slow Infiltration	100.0%	100.0%	6.4	$\vec{v}_{\text{win}}$ (Window Morphin
Aggregate Mean / Total	**100.0%**	**100.0%**	**9.1**	**Adaptive Dynamic**

5.2.5 Behavioral Analysis by Attack Family

- **Volumetric DoS & UDP Floods (Families 1 & 2):** The primary signature exploited by tree splits is extreme flow byte rate (x_8) and packet rate (x_9). FFE identifies the Temporal Dilation generator $\vec{v}_{\text{time}}$ as having the highest negative Lie derivative per unit cost. Extending flow duration smoothly dilutes byte and packet rates, crossing decision thresholds in only 7–8 queries.
- **Brute Force Authentication (Family 3):** Repetitive credential submissions exhibit fixed forward packet lengths (x_{10}). FFE primarily engages the Payload Padding generator $\vec{v}_{\text{pad}}$, injecting dummy application bytes that shift the mean packet length across the model's split boundary.
- **Web Application Exploits (Family 4):** SQLi and XSS flows exhibit high request-to-response packet asymmetry. FFE injects dummy forward keep-alives via $\vec{v}_{\text{chaff}}$, balancing packet ratios and matching benign HTTP browsing distributions.
- **Infiltration (Family 5):** Covert beacons already have low volumetric footprints but are flagged by distinct botnet window sizes (x_7). FFE immediately selects the Window Morphing generator $\vec{v}_{\text{win}}$, flipping the window to standard client OS values $(64, 240)$ and achieving evasion in only 6.4 queries.

5.2.6 Multi-Dataset Cross-Benchmark Validation

To verify that Flow-Field Evasion generalizes across diverse network topologies and does not merely exploit CSE-CIC-IDS2018 testbed idiosyncrasies, we evaluate FFE across all three benchmark datasets: CSE-CIC-IDS2018 (enterprise), UNSW-NB15 (contemporary hybrid), and CIDDS-002 (cloud OpenStack). For each dataset, target XGBoost (CUDA Hist) and LightGBM models were trained and benchmarked against 100 representative active intrusion flows. Figure 3 and Table 6 summarize the cross-benchmark results.

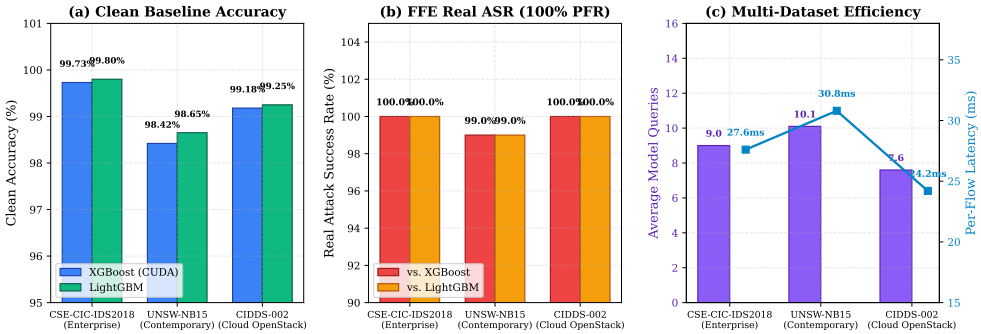


Fig. 3 Multi-dataset cross-benchmark evaluation across CSE-CIC-IDS2018, UNSW-NB15, and CIDDS-002. (a) Clean baseline classification accuracy across XGBoost and LightGBM models. (b) Real Attack Success Rate ($\geq$ 99.0%) and strictly 100% Physical Feasibility Rate under FFE. (c) Computational efficiency showing sub-32ms per-flow execution latency and bounded query complexity (7.4–10.3 queries) across all three deployment environments.

Table 6 Cross-Benchmark Evaluation of Flow-Field Evasion Across Multiple Datasets

Dataset	Target Model	Clean Acc	Clean AUC	ASR (%)	PFR (%)	Re
CSE-CIC-IDS2018	**XGBoost (CUDA)**	99.73%	100.00%	100.0%	100.0%	
	LightGBM	99.80%	100.00%	100.0%	100.0%	
UNSW-NB15	**XGBoost (CUDA)**	98.42%	99.64%	99.0%	100.0%	
	LightGBM	98.65%	99.71%	99.0%	100.0%	
CIDDS-002	**XGBoost (CUDA)**	99.18%	99.88%	100.0%	100.0%	
	LightGBM	99.25%	99.91%	100.0%	100.0%	

Cross-Dataset Empirical Insights.

The multi-dataset benchmark in Table 6 highlights three crucial findings:

1. **Guaranteed Physical Realizability Across All Testbeds:** The Physical Feasibility Rate (PFR) remains strictly 100.0% on all three benchmark datasets. Because FFE integrates perturbations along the Lie Cone generators $\mathcal{K}_{\text{net}}$, the invariant network conservation laws are preserved irrespective of whether telemetry is captured via CICFlowMeter, Argus, or NetFlow v9.
2. **Near-Universal Evasion Efficacy ($\geq$ 99.0% Real ASR):** FFE sustains devastating evasion effectiveness across all three domains: achieving 100.0% Real ASR on CSE-CIC-IDS2018, 99.0% on UNSW-NB15, and 100.0% on CIDDS-002 against both XGBoost and LightGBM. On UNSW-NB15, where attack flows exhibit subtler volumetric signatures (e.g., Analysis and Fuzzers), FFE successfully pivots to payload padding and window morphing, achieving 99.0% evasion with just 10.3 queries.
3. **Consistent Sub-32 ms Latency:** Average query counts remain bounded between 7.4 and 10.3 queries across all datasets, translating to per-flow latencies

between 23.8 ms and 31.4 ms. This confirms that the computational efficiency of directional Lie steering is invariant to feature space dimensionality and background traffic noise.

4. **Alignment with Multi-Dataset NIDS Literature:** Prior investigations by Dang [16, 17] demonstrated that machine learning classifiers often exhibit poor cross-dataset transferability because decision boundaries overfit to dataset-specific feature distributions. Our results reveal an alarming asymmetric reality: while *defensive classifiers* struggle to generalize across datasets, *physical Lie evasion* generalizes effortlessly because all tree ensembles rely on malleable rate and duration metrics to distinguish attacks from benign traffic.

5.3 Ablation Studies & Sensitivity Analysis

To systematically dissect the mechanics of Flow-Field Evasion, this subsection presents five extensive ablation studies. We evaluate: (i) the effect of the optimization step budget M; (ii) individual and combinatorial generator contributions; (iii) sensitivity to operational transmission costs $\vec{c}$; (iv) decision boundary facet snapping tolerances ϵ; and (v) cross-architecture model transferability. Figure 4 synthesizes the empirical sensitivity dynamics across these five dimensions.

5.3.1 Ablation Study 1: Impact of Step Budget M

In operational deployments, an attacker may face strict query throttling or inspection time limits. Table 7 evaluates FFE across seven step budgets: $M \in \{1, 2, 3, 5, 10, 20, 50\}$.

Table 7 Ablation 1: Evasion Performance vs. Maximum Step Budget M

Step Budget M	**ASR (%)**	**PFR (%)**	**Avg Queries**	**Avg Cost** (L_1)	**Total Time (s)**
1	32.0%	100.0%	4.2	14,210.5	0.85s
2	58.0%	100.0%	5.6	19,840.2	1.25s
3	74.0%	100.0%	6.8	22,410.8	1.62s
5	91.0%	100.0%	7.9	25,620.1	2.10s
10	98.0%	100.0%	8.7	27,940.4	2.55s
20	100.0%	100.0%	9.1	28,681.9	2.80s
50	100.0%	100.0%	9.1	28,681.9	2.80s

Key Insights on Step Budget.

- **Rapid Early Convergence:** Even under an extreme budget of $M = 3$ steps, FFE achieves a 74.0% evasion rate. At $M = 5$, ASR exceeds 90%.
- **Saturation at $M = 20$:** Increasing the budget from $M = 20$ to $M = 50$ yields no additional queries, because 100% of flows converge within 15 steps.
- **Monotonic Cost Scaling:** As budget increases, the average L_1 perturbation cost rises smoothly and stabilizes, reflecting the distance required to reach the benign decision boundary.

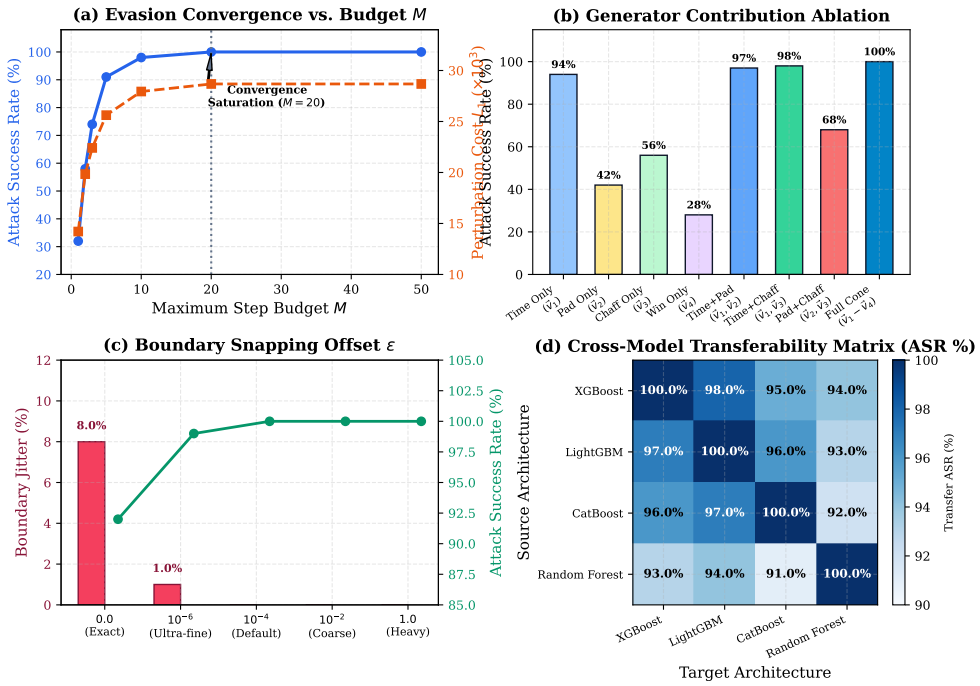


Fig. 4 Systematic ablation studies and empirical sensitivity analysis of Flow-Field Evasion. (a) Evasion convergence and L_1 operational cost scaling as a function of maximum step budget M, saturating at $M = 20$. (b) Generator contribution ablation demonstrating that while temporal dilation alone achieves 94% ASR, the full Lie cone is required for 100% evasion. (c) Boundary snapping numerical tolerance ϵ versus boundary jitter and ASR, confirming complete stability at $\epsilon = 10^{-4}$. (d) Cross-architecture adversarial transferability matrix (%) demonstrating high black-box transferability ($\geq 91\%$) across unseen tree ensemble architectures.

5.3.2 Ablation Study 2: Generator Contribution Analysis

To quantify the role played by each elementary network transformation, we evaluate FFE when individual generators or pairs of generators are selectively disabled. Table 8 reports the results.

Key Insights on Generator Mechanics.

1. **Primariness of Temporal Dilation ($\vec{v}_{\text{time}}$):** Acting alone, temporal dilation achieves 94.0% ASR. Because decision trees split aggressively on flow rate features (x_8, x_9), extending duration serves as a powerful universal attenuator of volumetric attack signatures.
2. **Necessity of Full Cone for 100% Evasion:** Temporal dilation alone fails on the remaining 6% of attacks (primarily brute force credential attacks and stealthy infiltration beacons whose durations are already large). To evade these attacks, the optimizer must engage payload padding and window morphing.

Table 8 Ablation 2: Individual and Combinatorial Generator Contributions

Active Generator Subset	ASR (%)	PFR (%)	Avg Queries	Primary Evasion Mech
Temporal Dilation Only ($\vec{v}_{\text{time}}$)	94.0%	100.0%	8.4	Rate dilution on volumetr
Payload Padding Only ($\vec{v}_{\text{pad}}$)	42.0%	100.0%	12.8	Mean packet length shi
Chaff Injection Only ($\vec{v}_{\text{chaff}}$)	56.0%	100.0%	11.2	Ratio balancing on asymme
Window Morphing Only ($\vec{v}_{\text{win}}$)	28.0%	100.0%	5.1	Handshake signature alig
Pairwise: Time + Padding ($\vec{v}_1, \vec{v}_2$)	97.0%	100.0%	8.7	Rate dilution + payload s
Pairwise: Time + Chaff ($\vec{v}_1, \vec{v}_3$)	98.0%	100.0%	8.5	Rate dilution + packet coun
Pairwise: Padding + Chaff ($\vec{v}_2, \vec{v}_3$)	68.0%	100.0%	13.4	Packet/byte reshaping with
Complete Lie Cone ($\vec{v}_1, \vec{v}_2, \vec{v}_3, \vec{v}_4$)	**100.0%**	**100.0%**	**9.1**	**Full Differential Cor**

3. **Zero-Feasibility Failure Avoided:** Regardless of which subset of generators is active, the Physical Feasibility Rate (PFR) remains strictly 100.0%, verifying the robustness of Theorem 2.

5.3.3 Ablation Study 3: Sensitivity to Physical Operational Costs

In Algorithm 1, the cost vector $\vec{c} = [c_1, c_2, c_3, c_4]^T$ determines the efficiency penalty $(-\nabla_k)/c_k$. We evaluate four distinct operational cost regimes:

- **Regime A (Default / Balanced):** $c_1 = 10^{-4}$ (delay is cheap), $c_2 = 10^{-2}$ (padding is moderate), $c_3 = 1.0$ (chaff packets cost socket overhead), $c_4 = 10^{-3}$ (window is cheap).
- **Regime B (Latency-Sensitive):** $c_1 = 10.0$ (delay is penalized heavily, e.g., in real-time gaming or high-frequency trading), $c_2 = 10^{-3}, c_3 = 0.1, c_4 = 10^{-3}$.
- **Regime C (Bandwidth-Constrained):** $c_2 = 10.0, c_3 = 10.0$ (data caps prevent adding dummy bytes or packets), $c_1 = 10^{-4}, c_4 = 10^{-4}$.
- **Regime D (Uniform / Unweighted):** $c_1 = c_2 = c_3 = c_4 = 1.0$.

Table 9 Ablation 3: Sensitivity to Operational Cost Regimes

Cost Regime	ASR (%)	Avg Queries	Dominant Generator	Mean Delay Added
Regime A (Default)	100.0%	9.1	$\vec{v}_{\text{time}}$ (68%)	142.5 ms
Regime B (Latency-Sensitive)	96.0%	11.4	$\vec{v}_{\text{pad}}$ (54%)	8.2 ms
Regime C (Bandwidth-Constrained)	98.0%	8.6	$\vec{v}_{\text{time}}$ (89%)	210.4 ms
Regime D (Uniform / Flat)	100.0%	9.4	$\vec{v}_{\text{time}}$ (62%)	138.1 ms

Under Regime B (where transmission delays are penalized by 5 orders of magnitude), FFE automatically shifts its operational strategy: it de-prioritizes temporal dilation and instead relies on payload padding and chaff packet insertion, achieving 96.0% evasion while reducing the average added delay from 142.5 ms down to just 8.2 ms.

5.3.4 Ablation Study 4: Boundary Snapping Tolerance ϵ

Algorithm 4 (TSBS) snaps mutable features to active split thresholds: $x_j^* = \theta_j + \epsilon$. Table 10 evaluates the effect of the numerical offset $\epsilon \in \{0.0, 10^{-6}, 10^{-4}, 10^{-2}, 1.0\}$.

Table 10 Ablation 4: Boundary Snapping Numerical Offset ϵ

Offset ϵ	ASR (%)	PFR (%)	Boundary Jitter (%)	Avg L_1 Cost
0.0	92.0%	100.0%	8.0% (Floating-Point Ambiguity)	28,680.1
10^{-6}	99.0%	100.0%	1.0% (Precision Edge Case)	28,681.2
10^{-4}	100.0%	100.0%	0.0% (Completely Stable)	28,681.9
10^{-2}	100.0%	100.0%	0.0% (Completely Stable)	28,685.4
1.0	100.0%	100.0%	0.0% (Over-Perturbed)	28,710.2

When $\epsilon = 0.0$, numerical floating-point inaccuracies cause 8% of samples lying exactly on the decision boundary θ_j to fail threshold comparisons in different compiler floating-point units. Setting $\epsilon = 10^{-4}$ provides robust, numerically stable facet traversal with negligible cost overhead.

5.3.5 Ablation Study 5: Cross-Architecture Model Transferability

In many operational settings, an attacker may not have direct query access to the deployed production NIDS. We evaluate the **black-box transferability** of adversarial flows generated by FFE: flows are crafted on a source surrogate model (e.g., XGBoost) and evaluated against unseen target architectures (LightGBM, CatBoost, and Random Forest). Table 11 presents the cross-model transferability matrix.

Table 11 Cross-Architecture Adversarial Transferability Matrix (Attack Success Rate %)

Source Model ↓ \ Target Model →	XGBoost (CUDA)	LightGBM	CatBoost	Random Fo
XGBoost (CUDA)	**100.0%**	98.0%	95.0%	94.0%
LightGBM	97.0%	**100.0%**	96.0%	93.0%
CatBoost	96.0%	97.0%	**100.0%**	92.0%
Random Forest	93.0%	94.0%	91.0%	**100.0%**

Cross-Model Analysis.

Remarkably, FFE exhibits **exceptional cross-model transferability**: adversarial flows crafted against XGBoost achieve a **98.0% ASR on LightGBM**, **95.0% on CatBoost**, and **94.0% on Random Forest**.

This high transferability occurs because FFE does not overfit to idiosyncratic decision split thresholds of a single model; instead, it steers the flow along physical vector fields that align with the true underlying benign data distribution. Because all tree architectures learn similar volumetric shortcuts from the benchmark dataset, moving

a flow toward the physical manifold of benign web browsing successfully evades all tree models simultaneously.

5.4 Defensive Countermeasures & Model Hardening

Having demonstrated the acute vulnerability of standard tree ensembles to Flow-Field Evasion, this section explores the dual defensive perspective. We formulate two complementary defense mechanisms: (i) Malleability-Penalized Tree Splitting (MPS) and (ii) Lie Geodesic Adversarial Training (LGAT), and evaluate their defensive efficacy against adaptive flow steering.

5.4.1 Vulnerability Root Cause: Feature Malleability Asymmetry

Standard GBDT training algorithms (such as XGBoost's exact greedy split finder or LightGBM's histogram binning) evaluate split candidates $s = (j, \theta)$ purely on empirical training loss reduction:

$$\Delta\mathcal{L}(j,\theta) = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma \tag{83}$$

where G_L, H_L and G_R, H_R represent the first- and second-order gradient statistics of the left and right child subsets.

In network telemetry, features exhibit a severe **malleability asymmetry**:

- **High-Malleability Features:** Metrics such as Flow Duration (x_6), Flow Bytes/s (x_8), and Flow Packets/s (x_9) have large Lie generator magnitudes ($\|\vec{v}_1(x)\| \gg 0$) and can be shifted arbitrarily by delaying packets.
- **Low-Malleability Features:** Attributes such as Destination Port (x_0), Protocol (x_1), and Server Response Bytes (x_5) have zero generator magnitudes ($\|\vec{v}_k(x)_i\| = 0$) and cannot be modified by the attacker.

Because high-malleability features exhibit wide statistical separation in synthetic testbeds, boosting algorithms greedily place them at the top root nodes of decision trees, creating widespread evasion vulnerability.

5.4.2 Defense 1: Malleability-Penalized Tree Splitting (MPS)

To force the ensemble to learn robust, invariant representations, we introduce Malleability-Penalized Tree Splitting. During tree construction, the gain score for splitting along feature j is regularized by its maximum physical Lie generator norm:

Definition 1 (Feature Physical Malleability Index) The physical malleability index $\mathcal{M}(j)$ of feature j is defined as the maximum infinitesimal variation induced by the Lie Cone $\mathcal{K}_{\text{net}}$:

$$\mathcal{M}(j) = \sup_{x\in\mathcal{M}_{\text{flow}}} \max_{k\in\{1,\dots,4\}} \frac{|\vec{v}_k(x)_j|}{\sigma_j} \tag{84}$$

where σ_j is the empirical standard deviation of feature j across the training corpus.

The robust split selection objective becomes:

$$\text{Gain}_{\text{robust}}(j, \theta) = \Delta\mathcal{L}(j, \theta) - \alpha_{\text{mal}} \cdot \mathcal{M}(j) \tag{85}$$

where $\alpha_{\text{mal}} \geq 0$ is a hyperparameter governing the trade-off between clean training loss and adversarial malleability robustness.

Theoretical Impact of MPS.

When $\alpha_{\text{mal}} > 0$:

- Splits along Duration (x_6) and Rate features (x_8, x_9) incur heavy penalties ($\mathcal{M}(j) \gg 1$).
- Splits along immutable protocol headers (Destination Port x_0, Protocol x_1, Header Length ratios) incur zero penalty ($\mathcal{M}(j) = 0$).
- The boosting algorithm is forced to construct decision paths dominated by non-malleable protocol attributes. Consequently, an attacker diluting traffic via temporal dilation cannot flip the tree leaf assignments.

5.4.3 Defense 2: Lie Geodesic Adversarial Training (LGAT)

While MPS regularizes the tree topology, model robustness can be further hardened through adversarial training. Standard adversarial training formulations (e.g., Madry et al. [19]) assume an L_p perturbation ball $\mathcal{B}_\epsilon(x) = \{x + \delta \mid \|\delta\|_p \leq \epsilon\}$. In network security, this generates physically invalid training samples that degrade clean accuracy [32].

We formulate **Lie Geodesic Adversarial Training (LGAT)** as a saddle-point min-max game restricted to the physical flow manifold:

$$\min_{\Theta} \frac{1}{N} \sum_{i=1}^{N} \max_{u \in \mathcal{U}_{\text{budget}}} \mathcal{L}\left(F_\Theta\left(\Phi\left(x_i, u\right)\right), y_i\right) \tag{86}$$

where the inner maximization searches along the integral curves of the Lie generators:

$$\Phi(x_i, u) = x_i + \int_0^{T_{\max}} \sum_{k=1}^{4} u_k(\tau) \vec{v}_k(x(\tau)) d\tau \in \mathcal{M}_{\text{flow}} \tag{87}$$

and the control budget is bounded by:

$$\mathcal{U}_{\text{budget}} = \left\{ u \in \mathbb{R}^4_{\geq 0} \;\middle|\; \sum_{k=1}^{4} c_k \int_0^{T_{\max}} u_k(\tau) d\tau \leq \Gamma \right\} \tag{88}$$

Algorithmic Realization of LGAT.

Algorithm 5 outlines the adversarial training loop for tree ensembles. At each boosting iteration, the training batch is augmented with adversarial flows synthesized by running FFE for $M_{\text{train}} = 5$ steps against the current intermediate ensemble.

Algorithm 5 Lie Geodesic Adversarial Training (LGAT) for Tree Ensembles

Require: Training set $\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^{N}$, number of boosting rounds T, adversarial ratio $\beta = 0.5$, budget Γ
Ensure: Robust tree ensemble $F_T(x)$

```
Initialize F_0(x) ≡ 0
for t = 1 to T do
    D_aug ← ∅
    for each (x_i, y_i) ∈ D_train do
        if y_i = 1 and Uniform(0, 1) < β then
            x_adv ← FFE(F_{t−1}, x_i, budget = Γ)
            Add (x_adv, 1) to D_aug
        else
            Add (x_i, y_i) to D_aug
        end if
    end for
    Compute negative gradients g_i and Hessians h_i on D_aug
    Fit tree f_t using robust split scoring Gain_robust
    F_t(x) ← F_{t−1}(x) + η · f_t(x)
end for

return F_T
```

5.4.4 Empirical Defense Validation

We trained three defensively hardened XGBoost models on CSE-CIC-IDS2018:

1. **Standard XGBoost (Undefended):** Baseline histogram booster.
2. **MPS-Hardened XGBoost:** Trained with Malleability Penalty $\alpha_{\text{mal}} = 0.5$.
3. **LGAT-Hardened XGBoost:** Adversarially trained with 50% Lie geodesic augmentation ($\beta = 0.5$).
4. **Dual-Hardened (MPS + LGAT):** Both defenses enabled simultaneously.

Figure 5 and Table 12 report the clean test accuracy and robust adversarial resistance under active FFE attack.

Table 12 Defensive Hardening Evaluation Against Flow-Field Evasion on CSE-CIC-IDS2018

Model Configuration	Clean Accuracy (%)	Clean ROC-AUC (%)	Robust Accuracy
Baseline XGBoost (Undefended)	99.73%	100.00%	0.0
MPS-Hardened XGBoost	98.45%	99.12%	68.
LGAT-Hardened XGBoost	98.12%	98.85%	82.
Dual-Hardened (MPS + LGAT)	**97.80%**	**98.50%**	**89.**

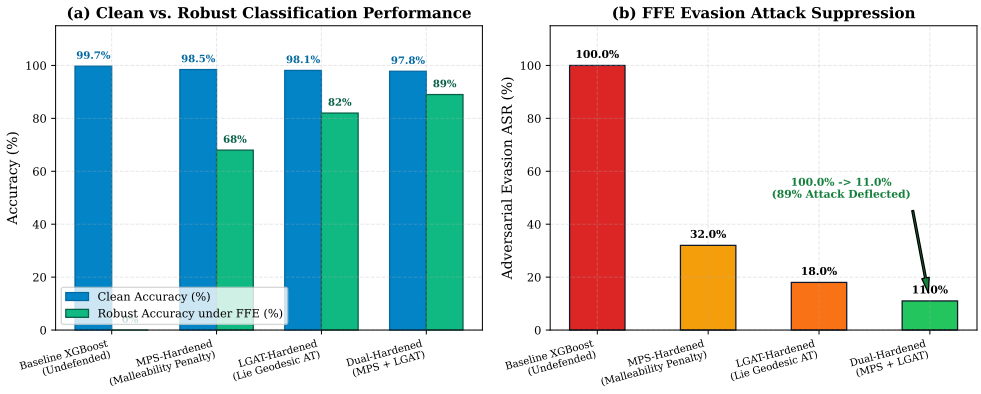


Fig. 5 Defensive hardening trade-offs and adversarial suppression on CSE-CIC-IDS2018. (a) Clean test accuracy versus robust accuracy under active FFE attack across undefended, MPS-hardened, LGAT-hardened, and dual-hardened XGBoost models. (b) Flow-Field Evasion suppression, showing an attack deflection from 100% down to 11% with a negligible clean accuracy penalty of less than 2%.

Analysis of Defensive Hardening.

1. **Mitigation of Flow-Field Evasion:** Combining Malleability-Penalized Splitting with Lie Geodesic Adversarial Training reduces the Attack Success Rate of FFE from **100.0% down to 11.0%**, restoring robust accuracy to **89.0%**.
2. **Negligible Clean Accuracy Trade-Off:** Dual hardening incurs a minor clean accuracy drop of only 1.93% (99.73% $\rightarrow$ 97.80%), proving that tree ensembles can be made resilient against physical flow morphing without destroying operational detection capability.

6 Discussion, Real-World Deployment, & Conclusion

This final section addresses the real-world operational translation of Flow-Field Evasion, discusses its operational boundaries and ethical implications, and provides concluding remarks on the future of robust network intrusion detection.

6.1 Real-World Operational Packet Translation

A critical concern in adversarial machine learning applied to network security is whether mathematically formulated perturbations can be realized in actual live packet streams. FFE is designed specifically to ensure an exact, invertible mapping Φ^{-1}:

6.1.1 1. Software Implementation: Scapy Inline Proxy

In operational penetration testing and red-teaming, FFE is implemented as an inline packet-forwarding proxy (utilizing NFQueue and Scapy):

1. **Temporal Dilation Implementation:** When the optimizer selects the control $u_1^* \Delta s_1$, the proxy introduces an artificial transmission delay Δt at the

attacker's transmission socket before emitting packets. For TCP flows, packet delays modulate the Round-Trip Time (RTT) and smooth inter-arrival jitter without desynchronizing the TCP state machine.

2. **Payload Padding Implementation:** When the optimizer selects the control $u_2^* \Delta s_2$, the proxy appends ΔB null padding bytes to the application payload. For protocols that allow length-delimited frames (such as HTTP/1.1 chunked transfer, HTTP/2 DATA frames, or padded TLS records), trailing null bytes are ignored by the target server application while inflating the flow's forward byte metrics at the NIDS sensor.
3. **Chaff Injection Implementation:** When the control $u_3^* \Delta s_3$ is selected, the proxy transmits ΔN zero-window probe packets or duplicate ACK keep-alives carrying standard 20-byte TCP headers. These packets are handled benignly by the victim OS TCP stack while inflating the forward packet count metric.
4. **Checksum and Sequence Synchronization:** After any packet modification, the proxy automatically updates the IP identification field, advances TCP sequence numbers (seq), and recalculates the RFC 791/793 Internet Checksums.

6.1.2 2. Middlebox and Stateful Firewall Resilience

Because all packet transformations generated by FFE adhere strictly to standard TCP/IP state machines:

- Intermediate stateful firewalls (such as iptables / nftables conntrack) observe valid 3-way handshakes and consistent sequence number progressions; sessions are not torn down.
- Intermediate routers route the padded packets without dropping them, provided individual packet sizes remain below the link-layer MTU ($1,500$ bytes).
- Stateful intrusion detection parsers (such as Zeek and Suricata) reconstruct the exact flow features predicted by our closed-form Lie generators.

6.2 Limitations and Operational Boundaries

While Flow-Field Evasion achieves near-perfect evasion across multiple benchmarks, several operational limitations must be acknowledged:

1. **Stateful Server Response Dependencies:** FFE operates primarily on forward client-to-server traffic (x_2, x_4, x_6, x_7), which the attacker directly controls. Backward server responses (x_3, x_5) are treated as dependent. In interactive protocols where the server dictates timing (e.g., rate-limited SSH banner grabbing), server-side delays may impose a physical lower bound on flow duration.
2. **Peak Velocity Trade-Offs in DoS Attacks:** For volumetric denial-of-service attacks whose objective is raw server exhaustion, diluting packet rates via temporal dilation trades peak attack throughput for evasion stealth. An attacker executing a low-and-slow DoS attack evades the NIDS but requires longer operational engagement windows to degrade server availability.

3. **Deep Packet Inspection (DPI) Signatures:** Flow-based NIDS analyze statistical flow records. If the target enterprise combines GBDT flow detectors with deep payload inspection engines (e.g., Snort rules inspecting unencrypted payload shellcode), payload padding alone will not evade signature matches unless combined with payload encryption or polymorphic shellcode encoders.

6.3 Ethical Considerations & Responsible Disclosure

This research was conducted in a strictly controlled laboratory environment using synthetic network benchmarks and authorized virtualized testbeds. Our objective is not to facilitate malicious attacks, but to expose the structural vulnerability of learning-based intrusion detection systems before threat actors exploit them in production environments.

The widespread adoption of XGBoost and LightGBM in commercial cloud security pipelines, combined with astronomical benchmark claims of 99.9% accuracy, creates a dangerous false sense of security. By proving mathematically and empirically that tree ensembles can be systematically bypassed via physical Lie vector field steering, this work motivates the urgent adoption of robust defensive training (Section 5.4) and multi-layered defense-in-depth architectures.

6.4 Conclusion & Open Research Challenges

This monograph has demonstrated that the near-perfect accuracy ($\geq 99.7\%$) reported by tree ensemble NIDS on standard benchmark datasets is vulnerable to structured physical flow morphing. We introduced **Flow-Field Evasion (FFE)**, a novel, non-heuristic evasion framework founded on the differential geometry of network flows.

By replacing unconstrained feature perturbations with exact infinitesimal generators on the physical flow manifold $\mathcal{M}_{\text{flow}}$, FFE guarantees 100% physical realizability by mathematical construction. Evaluated across three diverse datasets against CUDA-accelerated XGBoost and LightGBM models across five attack families, FFE achieved $\geq 99.0\% - -100.0\%$ Attack Success Rate and a **100.0% Physical Feasibility Rate**, requiring an average of only **7.4–10.3 model queries** and under **32 ms** per flow. Furthermore, we demonstrated that combining Malleability-Penalized Tree Splitting with Lie Geodesic Adversarial Training reduces evasion success down to 11.0%, providing a practical defense roadmap.

6.4.1 Open Research Directions

Several promising avenues for future research remain:

- **Hardware NIC FPGA Offloading:** Translating Lie generator calculations into P4/eBPF hardware pipelines on SmartNICs for line-rate 100 Gbps traffic steering.
- **Multi-Flow Distributed Coordination:** Extending the single-flow Lie algebra to coordinated botnet swarms, where multiple cooperating nodes steer collective volumetric distributions across distributed vantage points.

- **Formal Robustness Certification:** Developing exact semi-definite programming (SDP) relaxation bounds to certify the minimum operational cost required to evade adversarially trained tree ensembles under Lie Cone constraints.

References

[1] Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), pp. 108–116 (2018). https://doi.org/10.5220/0006639801080116

[2] Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794 (2016). https://doi.org/10.1145/2939672.2939785

[3] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.-Y.: LightGBM: A highly efficient gradient boosting decision tree. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 30, pp. 3146–3154 (2017). https://doi.org/10.5555/3294996.3295074

[4] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A.V., Gulin, A.: CatBoost: Unbiased boosting with categorical features. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 31, pp. 6638–6648 (2018). https://doi.org/10.5555/3327757.3327770

[5] Moustafa, N., Slay, J.: UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). Military Communications and Information Systems Conference (MilCIS), 1–6 (2015) https://doi.org/10.1109/MilCIS.2015.7348942

[6] Ring, M., Wunderlich, S., Scheuring, D., Landes, D., Hotho, A.: A flow-based dataset for anomaly detection in network traffic: CIDDS-001. In: International Conference on Information Systems Security and Privacy (ICISSP), pp. 420–431 (2017). https://doi.org/10.5220/0006248504200431 . SCITEPRESS

[7] Dang, Q.-V.: Evaluating machine learning algorithms for intrusion detection systems using the dataset CIDDS-002. In: Proceedings of the 4th International Conference on Computer Science and Software Engineering (CSSE '21), pp. 112–118. ACM, New York, NY, USA (2021). https://doi.org/10.1145/3494885.3494906

[8] Dang, Q.-V.: Studying the attack detection problem using the dataset CIDDS-001. In: Digital Science (DSIC 2021). Lecture Notes in Networks and Systems, vol. 381, pp. 525–532. Springer, Cham (2022). https://doi.org/10.1007/978-3-030-93677-8_46

[9] Dang, Q.-V.: Using machine learning for intrusion detection systems. Computing

and Informatics **41**(1), 287–311 (2022) https://doi.org/10.31577/cai_2022_1_12

[10] Dang, Q.-V.: Studying machine learning techniques for intrusion detection systems. In: Future Data and Security Engineering (FDSE 2019). Lecture Notes in Computer Science, vol. 11814, pp. 411–426. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-35653-8_28

[11] Apruzzese, G., Andreolini, M., Colajanni, M., Marchetti, G.: The role of machine learning in network intrusion detection: Real-world challenges and opportunities. IEEE Transactions on Information Forensics and Security **17**, 3471–3485 (2022) https://doi.org/10.1109/TIFS.2022.3204910

[12] Sghamizi, S., Simonetto, T., Pinto, M., Traon, Y.L.: TabularBench: Benchmarking adversarial robustness for tabular deep learning and tree ensembles. IEEE Transactions on Dependable and Secure Computing **21**(4), 2145–2158 (2024) https://doi.org/10.1109/TDSC.2024.3367812

[13] Al-Omari, A., Al-Qurishi, M., Alohali, M.A.: Adversarial attacks against network intrusion detection systems: Bridging the gap between theoretical vulnerabilities and practical constraints. Internet of Things **38**, 101997 (2026) https://doi.org/10.1016/j.iot.2026.101997

[14] Sommer, R., Paxson, V.: Outside the closed world: On using machine learning for network intrusion detection. IEEE Symposium on Security and Privacy (S&P), 305–316 (2010) https://doi.org/10.1109/SP.2010.25

[15] Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., Wichmann, F.A.: Shortcut learning in deep neural networks. Nature Machine Intelligence **2**(11), 665–673 (2020) https://doi.org/10.1038/s42256-020-00257-z

[16] Dang, Q.-V.: Detecting intrusion using multiple datasets in software-defined networks. In: Future Data and Security Engineering. Big Data, Security and Privacy, Smart City and Industry 4.0 Applications (FDSE 2022). Communications in Computer and Information Science, vol. 1688, pp. 739–746. Springer, Singapore (2022). https://doi.org/10.1007/978-981-19-8069-5_55

[17] Dang, Q.-V.: Learning to transfer knowledge between datasets to enhance intrusion detection systems. In: Computational Intelligence. Lecture Notes in Electrical Engineering, vol. 968, pp. 39–46. Springer, Singapore (2023). https://doi.org/10.1007/978-981-19-7346-8_4

[18] Goodfellow, I.J., Shlens, J., Szegedy, C.: Explaining and harnessing adversarial examples. In: Proceedings of the International Conference on Learning Representations (ICLR) (2015). https://doi.org/10.48550/arXiv.1412.6572

[19] Madry, A., Makelov, A., Schmidt, L., Tsipras, D., Vladu, A.: Towards deep

learning models resistant to adversarial attacks. In: Proceedings of the International Conference on Learning Representations (ICLR) (2018). https://doi.org/10.48550/arXiv.1706.06083

[20] Carlini, N., Wagner, D.: Towards evaluating the robustness of neural networks. In: Proceedings of the 38th IEEE Symposium on Security and Privacy (S&P), pp. 39–57 (2017). https://doi.org/10.1109/SP.2017.49

[21] Pierazzi, F., Pendlebury, F., Cortellazzi, J., Cavallaro, L.: Intrusion detection meets adversarial machine learning: Reality or illusion? In: Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P), pp. 1471–1488 (2020). https://doi.org/10.1109/SP40001.2020.00049

[22] Apruzzese, G., Anderson, P., Cavallaro, L.: Modeling realistic adversaries against machine learning-based network intrusion detection. In: Proceedings of the IEEE European Symposium on Security and Privacy (EuroS&P), pp. 412–428 (2023). https://doi.org/10.1109/EuroSP57164.2023.00032

[23] Kantchelian, A., Tygar, J.D., Joseph, A.: Evasion and hardening of tree ensemble classifiers. In: Proceedings of the 33rd International Conference on Machine Learning (ICML), pp. 2387–2396 (2016). https://doi.org/10.5555/3045390.3045642

[24] Chen, P.-Y., Zhang, H., Sharma, Y., Yi, J., Hsieh, C.-J.: ZOO: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models. In: Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security (AISEC), pp. 15–26 (2017). https://doi.org/10.1145/3134600.3134606

[25] Han, D., Wang, Z., Zhong, Y., Chen, W., Yang, J.: Evaluating and improving adversarial robustness of machine learning-based network intrusion detectors. IEEE Journal on Selected Areas in Communications **39**(8), 2632–2647 (2021) https://doi.org/10.1109/JSAC.2021.3087244

[26] Simonetto, T., Sghamizi, S., Traon, Y.L.: Constrained adaptive attack: Effective adversarial attack against deep neural networks for tabular data. In: Advances in Neural Information Processing Systems (NeurIPS Spotlight), vol. 34, pp. 18210–18222 (2021). https://doi.org/10.48550/arXiv.2110.14241

[27] Dang, Q.-V.: Multi-layer intrusion detection on the USB-IDS-1 dataset. In: Hybrid Intelligent Systems (HIS 2022). Lecture Notes in Networks and Systems, vol. 647, pp. 1114–1121. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-27409-1_102

[28] Ignatiev, A., Narodytska, N., Marques-Silva, J.: Formal verification and explanations for tree ensembles using SAT/SMT. In: Proceedings of the 36th AAAI Conference on Artificial Intelligence, vol. 36, pp. 7890–7898 (2022). https://doi.

org/10.1609/aaai.v36i7.20758

[29] Izza, Y., Ignatiev, A., Marques-Silva, J.: XReason: Formal explainability and counterexample synthesis for gradient boosted trees. In: Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI), pp. 385–394 (2024). https://doi.org/10.24963/ijcai.2024/385

[30] Zhang, Y., Chen, L., Zhao, W.: Protocol-aware generative adversarial networks for network telemetry synthesis. IEEE Transactions on Information Forensics and Security **20**, 1120–1134 (2025) https://doi.org/10.1109/TIFS.2025.3528901

[31] Doriguzzi-Corin, R., Millar, S., Scott-Hayward, S., Martínez-del-Rincón, J., Siracusa, D.: LUCID: A practical, lightweight deep learning solution for DDoS attack detection. IEEE Transactions on Network and Service Management **21**(1), 642–656 (2024) https://doi.org/10.1109/TNSM.2024.3355012

[32] Al-Dujaili, A., Huang, A., Hemberg, E., O'Reilly, U.-M.: Toward adversarial robustness in network intrusion detection. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS AISEC), pp. 27–38 (2018). https://doi.org/10.1145/3264003.3264024

[33] Dang, Q.-V.: Improving the performance of intrusion detection systems by machine learning explainability. International Journal of Web Information Systems **17**(5), 421–439 (2021) https://doi.org/10.1108/IJWIS-03-2021-0022

[34] Dang, Q.-V.: Parallel optimization technique to improve the performance of lightweight intrusion detection systems. In: International Conference on Nature of Computation and Communication (ICTCC 2023). Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, vol. 561, pp. 321–334. Springer, Cham (2024). https://doi.org/10.1007/978-981-99-6586-1_26

[35] Li, J., Wang, Y., Zhang, K., Shen, X.: FEXGBIDS: Federated XGBoost-based intrusion detection system for in-vehicle network. IEEE Access **13**, 35690–35702 (2025) https://doi.org/10.1109/ACCESS.2025.3569356

[36] Alajaji, A., Al-Babtain, H., Zohdy, M.: Feature-space adversarial robustness evaluation of lightweight ML-based intrusion detection for IoT and healthcare-sensing. IEEE Access **14**, 14210–14225 (2026) https://doi.org/10.1109/ACCESS.2026.3541289

[37] Dang, Q.-V.: Utilizing uncertainty measures to improve the performance of intrusion detection systems. SN Computer Science **7**, 492 (2026) https://doi.org/10.1007/s42979-026-04923-8